DEVELOPER WALKTHROUGH

AgentOps

Enterprise Agentic P2P & Procurement Platform

Sixteen agents. Twenty-eight skills. Zero write access.

16	28	33	29	79
agents	skills	actions	tables	tests

Version 1.1.0 · 30 screenshots captured live from the running application · Companion to `docs/SPECIFICATION.md` and `agents/<key>/AGENT.md`

HOW THIS GUIDE RUNS

Contents

Three parts, in the order a new developer needs them.

Every screenshot in Part 02 was captured from this application running locally, after a live agent sweep against the seeded dataset. Nothing in this guide is a mock-up, and the terminal output in Part 03 is real output from the commands shown.

Companion documents

DOCUMENT	WHAT IT COVERS
`docs/SPECIFICATION.md`	The platform specification of record — guarantees, architecture, every enumerated value, the data model, governance, API, and a rebuild order
`agents/<key>/AGENT.md`	One complete build specification per agent. Enough to rebuild any agent without reading its source
`skills/<name>/SKILL.md`	The 28 shared capabilities and their contracts
`RUNNING.md`	Prerequisites, every launch path, configuration reference, troubleshooting

PART 01 · ORIENTATION

One rule explains the whole system

> ## An agent returns proposals, never mutations.
>
> There is no code path from an agent to a business table. This is not a convention or a review rule — it is a structural fact about how the code is shaped.

1 · A proposal is inert

`ProposedAction` is a dataclass. No database session, no ORM object, no executor reference. It cannot write because there is nothing in it to write with.

2 · One consumer

The only code that reads a proposal is `hitl.create_checkpoint()`, which writes a `HumanTask` row and nothing else.

3 · One writer

Business data is written only by an executor registered with `@action(...)` in `services/hitl.py`, and the only caller of an executor is `hitl.execute_action()` — reached exclusively from `hitl.decide()`, after an authority check.

```
@dataclass
class ProposedAction:
    action_kind: str          # one of 33 ActionKinds
    title: str
    summary: str
    payload: dict             # what the executor will apply
    diff_preview: list[dict]  # before/after for the reviewer
    alternatives: list[dict]  # priced options, each with its own payload
    confidence: float
    financial_impact_usd: float
    artifact_ids: list[str]   # drafts this action releases on approval

# No session. No ORM object. No executor reference.
# It cannot write anything, because there is nothing here to write with.
```

Three consequences follow, and they are the reason the guarantee is worth having. An agent that wanted to write to a business table would have to be *rewritten* to do so, which means this is not a matter of developer discipline. A code reviewer does not have to scan an agent for stray writes. And testing the guarantee is trivial: run the agent, count the rows, assert nothing moved.

> **Check any change against this sentence, not the feature list.** An agent that can write to a business table gives you the same screens and none of the guarantees.

PART 01 · ORIENTATION

Where everything lives

Roughly 25,000 lines across a FastAPI backend and a React SPA.

```
AgentOps-EnterpriseAgentic-P2P-Platform/
├── backend/app/
│   ├── enums.py          roles, stages, 33 action kinds, IRREVERSIBLE_ACTIONS
│   ├── models.py         29 tables in seven domains
│   ├── agents/           base.py + 16 agents + registry + orchestrator
│   ├── skills/           28 analysis capabilities — no writes
│   ├── services/
│   │   ├── policy.py     the seven gates, default deny
│   │   ├── hitl.py       checkpoints + the ONLY business writes
│   │   ├── artifacts.py  draft → released lifecycle, parsers
│   │   └── audit.py      the hash chain
│   └── api/              auth · core · hitl · agents · procurement
│                         · analytics · admin · explain
├── frontend/src/         22 screens, React + TS + Tailwind
├── agents/<key>/AGENT.md 16 generated build specifications
├── skills/<name>/SKILL.md 28 generated skill contracts
├── docs/SPECIFICATION.md the platform specification of record
└── scripts/              the two documentation generators
```

The four files that carry the guarantees

FILE	WHY IT MATTERS
`services/policy.py`	The seven gates. Read this before changing any threshold — it is the only place that decides whether an action may bypass a human.
`services/hitl.py`	The only place business data is written. Thirty-three executors, one per action kind, each registered by decorator.
`agents/base.py`	The run loop every agent shares. Because it is shared, the guarantees hold uniformly rather than per agent.
`enums.py`	Roles, stages, action kinds, the irreversible set. Everything downstream references it; change here and regenerate the docs.

PART 01 · ORIENTATION

Three ways to run it

No cloud account, no API key, no database server. SQLite by default, one process, one port.

Standard — use this for a demo

```
./run.sh          # macOS / Linux / WSL
.\run.ps1         # Windows PowerShell
# → http://localhost:8000
```

Creates the virtualenv, installs dependencies, builds the SPA, seeds the demo dataset if the database is empty, and serves both the API and the UI on port 8000. Idempotent — safe to re-run at any time. First run takes one to three minutes; later runs start in seconds.

Development — hot reload

```
./run.sh --dev             # or:  .\run.ps1 -Dev
# UI  → http://localhost:5173   (Vite, hot module reload)
# API → http://localhost:8000   (Vite proxies /api to it)

P2P_RELOAD=1 ./run.sh --dev    # backend auto-reload too
```

Use port 5173 in this mode. Port 8000 serves the last *built* UI, which will be stale.

Docker

```
docker compose up --build
docker compose down -v && docker compose up --build   # fresh start
```

Nothing needed on the host. The demo database persists in the named volume `p2p-data`, so decisions survive a restart. The compose file also contains a commented-out PostgreSQL service — uncomment it and the `P2P_DATABASE_URL` line for a shared multi-user demo. The schema is identical either way; there is no migration step.

> Between client meetings, reset from the UI: sign in as **Platform Service Account** → **Governance** → **Rebuild**. Or `./run.sh --reset`.

PART 01 · ORIENTATION

The authority ladder

A ladder, not a set — a Controller can decide everything a clerk can, plus more. Enforced server-side on every decision, not in the UI.

ROLE	AUTHORITY	MAY DECIDE
`supplier`	**0**	Nothing — external party
`ap_clerk`	**10**	Extraction, matching, exceptions, supplier drafts
`procurement / treasury`	**20**	+ sourcing, savings, dispositions / payment scheduling
`ap_manager`	**30**	+ ERP posting, escalations, invoice approvals
`controller`	**40**	+ payment release, master data, contract signature
`cfo`	**50**	Everything, including executive briefs
`admin`	**60**	+ agent autonomy, the governance switch

Escalation by value

On top of the per-action minimum in `ACTION_MIN_ROLE`, the required approver rises with the money at stake:

FINANCIAL IMPACT	EFFECT	FLAG
≥ $50,000	Minimum approver raised to Controller	`controller_threshold`
≥ $250,000	Minimum approver raised to CFO	`cfo_threshold`
≥ $100,000 *and irreversible*	Two distinct approvers required	`dual_approval`

Dual approval compares `user.id`, not role. The same person approving twice is one approval. This is checked in `hitl.decide()`, which reads the prior approver off the task before accepting a second signature.

The dual-approval threshold itself is a policy row (`hitl.dual_approval_above`), so it can be tightened from the Governance screen without a deploy — and the change is audited.

PART 02

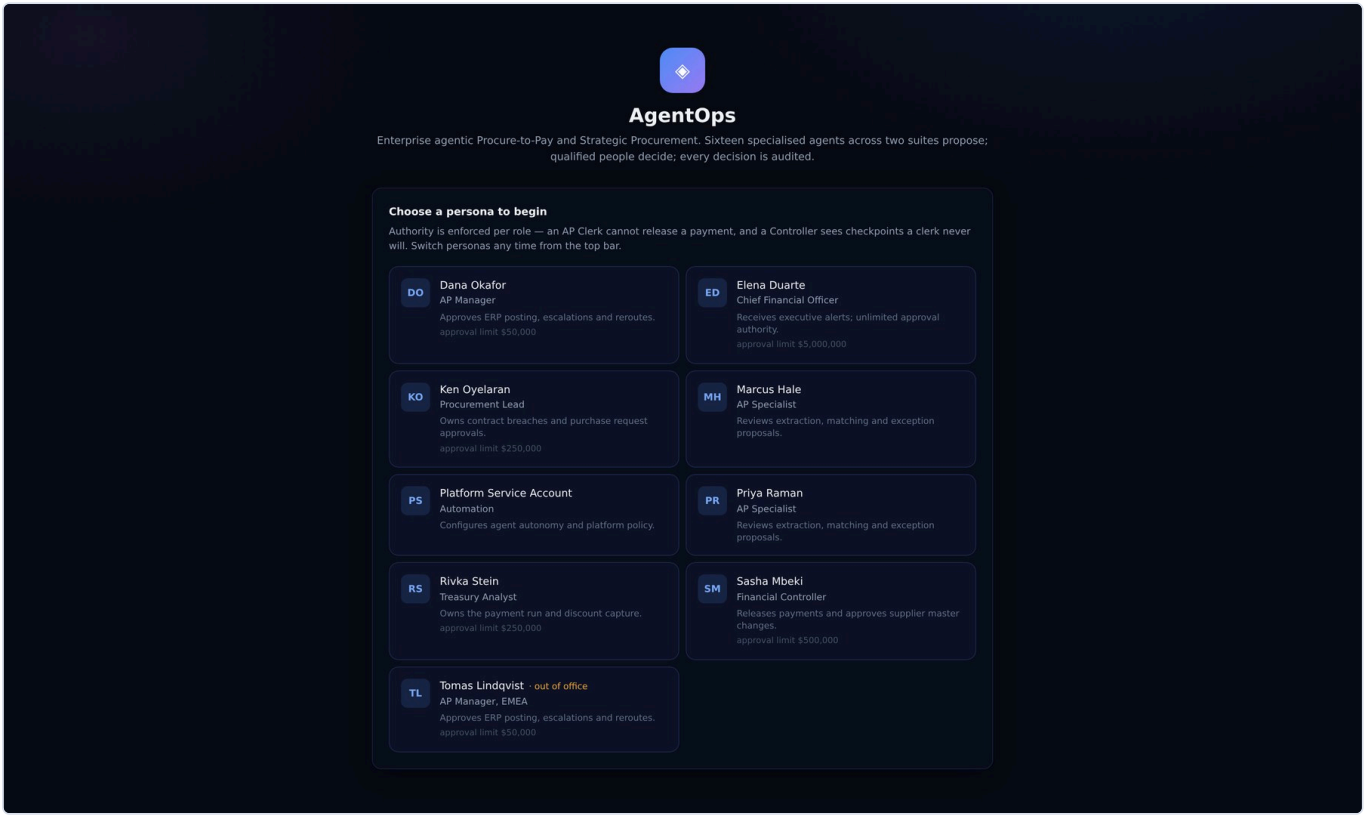
The product, screen by screen

All 22 screens, captured live after an agent sweep against the seeded dataset. For each: what it shows, why it is shaped that way, and what a developer should notice.

PART 02 · GETTING STARTED

Sign in — personas, not passwords

Nine seeded personas across the authority ladder. Identity is selected; authority is enforced.



There are no passwords. The login screen lists nine seeded personas spanning the authority ladder, and picking one sets a single header on every subsequent request. This is a deliberate demo affordance, and it is also exactly where a production deployment differs: swap the persona picker for SSO, keep `get_current_user`, and every authority check downstream is unchanged. What is *not* a demo affordance is the enforcement — what each persona may decide is checked server-side on every decision, and no UI state can loosen it.

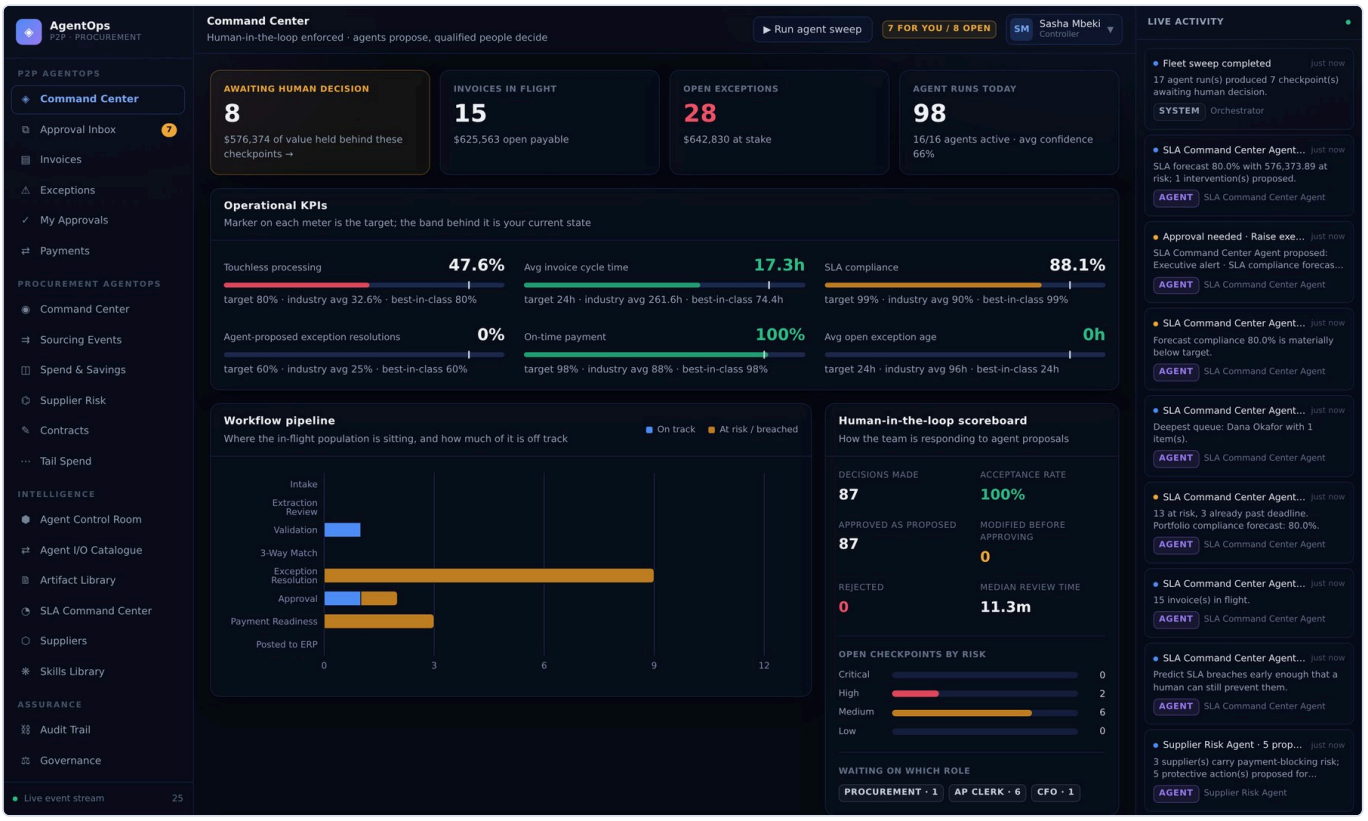
WHAT TO LOOK AT

1. **The persona *is* the auth token**
   The client sends the chosen id in `X-User-Id`; `get_current_user` resolves it. Production puts SSO here and changes nothing else.
2. **Role drives everything downstream**
   What each persona can *decide* is enforced server-side in `hitl.decide()`, not hidden in the UI.
3. **Start as Priya (AP Clerk)**
   The lowest rung makes the authority limits visible within two clicks.

PART 02 · P2P AGENTOPS

Command Center — the portfolio at a glance

KPIs against industry benchmarks, the invoice pipeline, aging, and the human-in-the-loop scoreboard.



The Command Center is the first screen a stakeholder sees, so it answers the two questions they arrive with: is this working, and what is waiting on me. The KPI row measures against published industry benchmarks rather than internal targets, which makes "80% touchless" mean something outside the room. The right-hand rail is a live Server-Sent Events stream — leave it open during a sweep and you watch each agent plan, call its tools and land its proposals in real time.

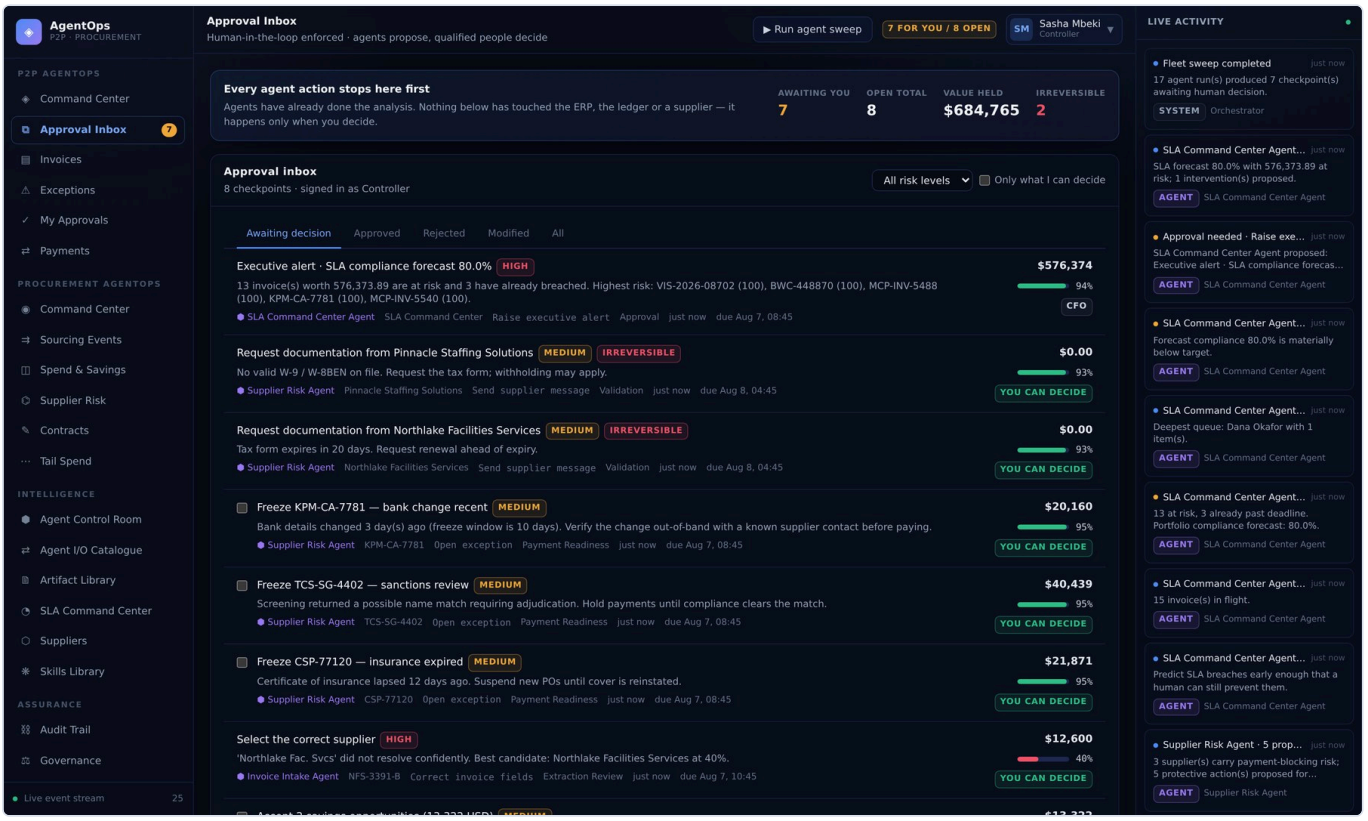
WHAT TO LOOK AT

1. **Touchless rate is real**
   Computed from `invoices.human_touches`, not a constant. Approving proposals moves it.
2. **The HITL scoreboard**
   Open checkpoints, value held, and how many are irreversible — the number that matters in a governance conversation.
3. **Live activity rail**
   Server-Sent Events from `GET /api/events/stream`, fed by `workflow_events`.

PART 02 · P2P AGENTOPS

Approval Inbox — where every agent action stops

Nothing below has touched the ERP, the ledger or a supplier. That happens only when a human decides.



This is the screen the platform exists to produce. Every proposal from every agent lands here and stops. The banner is literal rather than reassuring: at the moment you are reading the list, nothing in it has touched the ERP, the ledger or a supplier. Each row is one `human_tasks` record; the risk badge and the irreversible flag are the policy verdict computed when the proposal was created, stored on the row, and never recomputed — so what you see is what the engine actually decided.

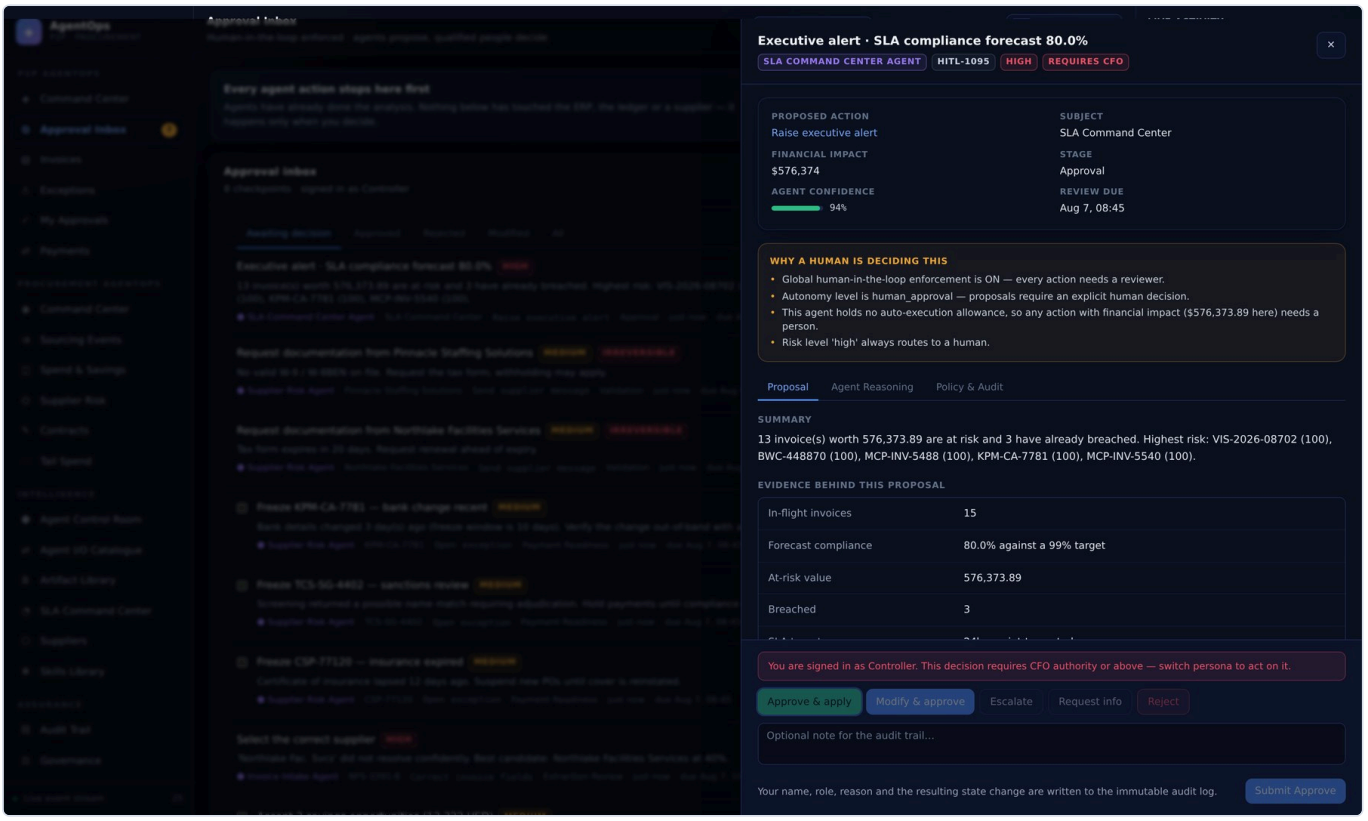
WHAT TO LOOK AT

1. **One row per `human_tasks` record**
   The inbox is a straight read of the checkpoint table, filtered by what your role may decide.
2. **Risk and irreversibility are badges**
   Both come from the policy verdict stored on the task at creation time.
3. **"Only what I can decide"**
   Toggles the role filter. Higher roles see *more* checkpoints, not fewer.

PART 02 · THE CHECKPOINT

Proposal — what would change, and why it stopped

The amber panel answers the auditor's question directly: why is a human deciding this?



Open a checkpoint and the amber panel is the first thing you read. It is generated from the stored `PolicyDecision`, listing the gates that *blocked* this action in the engine's own words. That is what makes the screen defensible in an audit: it is not an explanation written afterwards, it is the verdict itself. Note the red band above the buttons. Signed in as a Controller against a CFO-level decision, the platform tells you plainly that you cannot act — and if you posted the request anyway, `hitl.decide()` would refuse it.

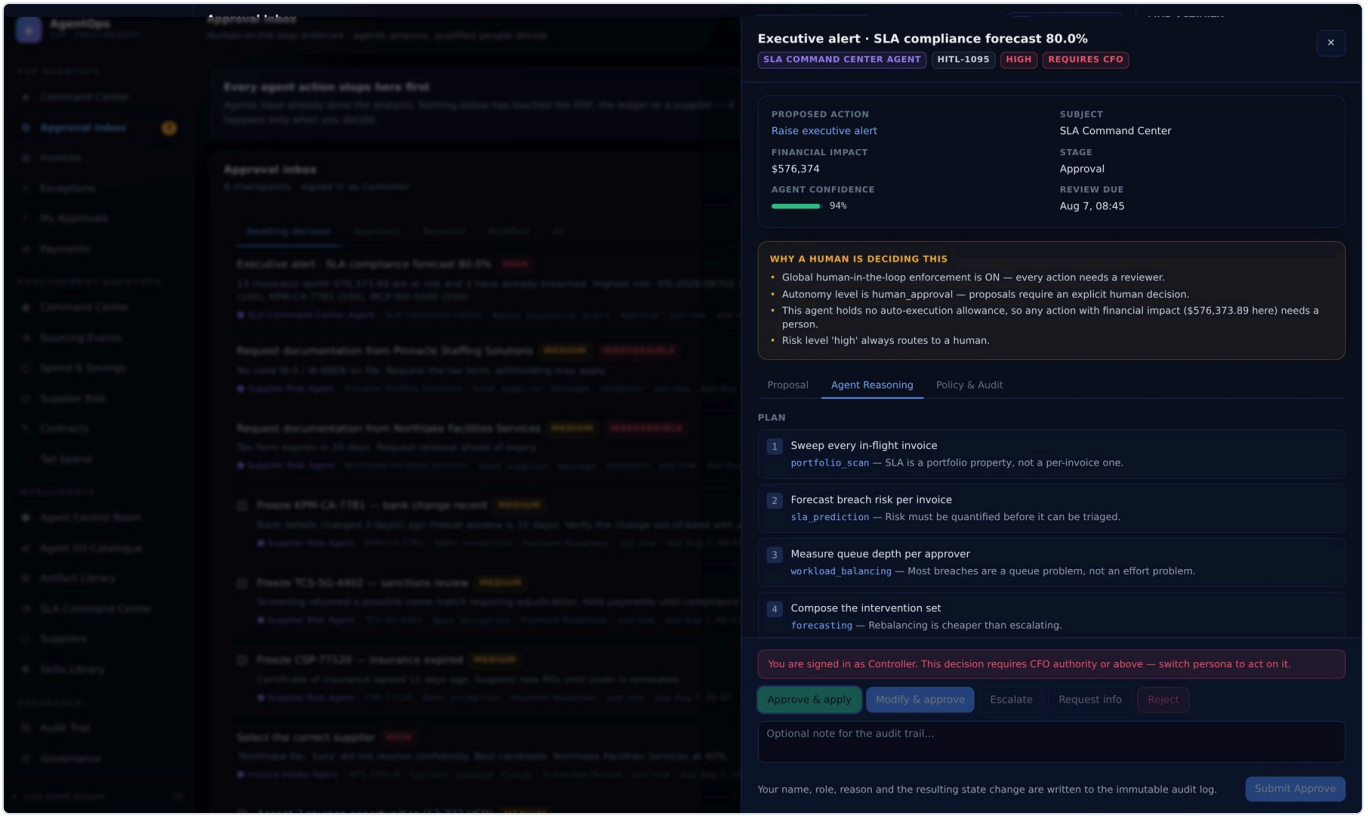
WHAT TO LOOK AT

1. **"Why a human is deciding this"**
   Each line is a policy gate that blocked, rendered from the stored `PolicyDecision`. Not marketing copy — the actual verdict.
2. **Authority is enforced here**
   Signed in as Controller, this CFO-level decision is refused. The buttons are present; the server rejects it.
3. **Evidence, not assertion**
   Every row cites the observation it came from, so the decision is reconstructable months later.

PART 02 · THE CHECKPOINT

Agent reasoning — the plan, the tools, the confidence

The full run: what it planned, each tool call and what came back, and the decision rules it applied.



The reasoning tab is the agent's working shown in full: the plan it declared before starting, every tool it called with what came back, the evidence it chose to cite, and the decision rules it applied. All of it is read from `agent_executions`, not regenerated — open this checkpoint in a year and it says the same thing. The decision rules are deliberately quantitative. "Amount variance +8.00% against a 3% tolerance" is a claim a supplier can dispute; "looks incorrect" is not.

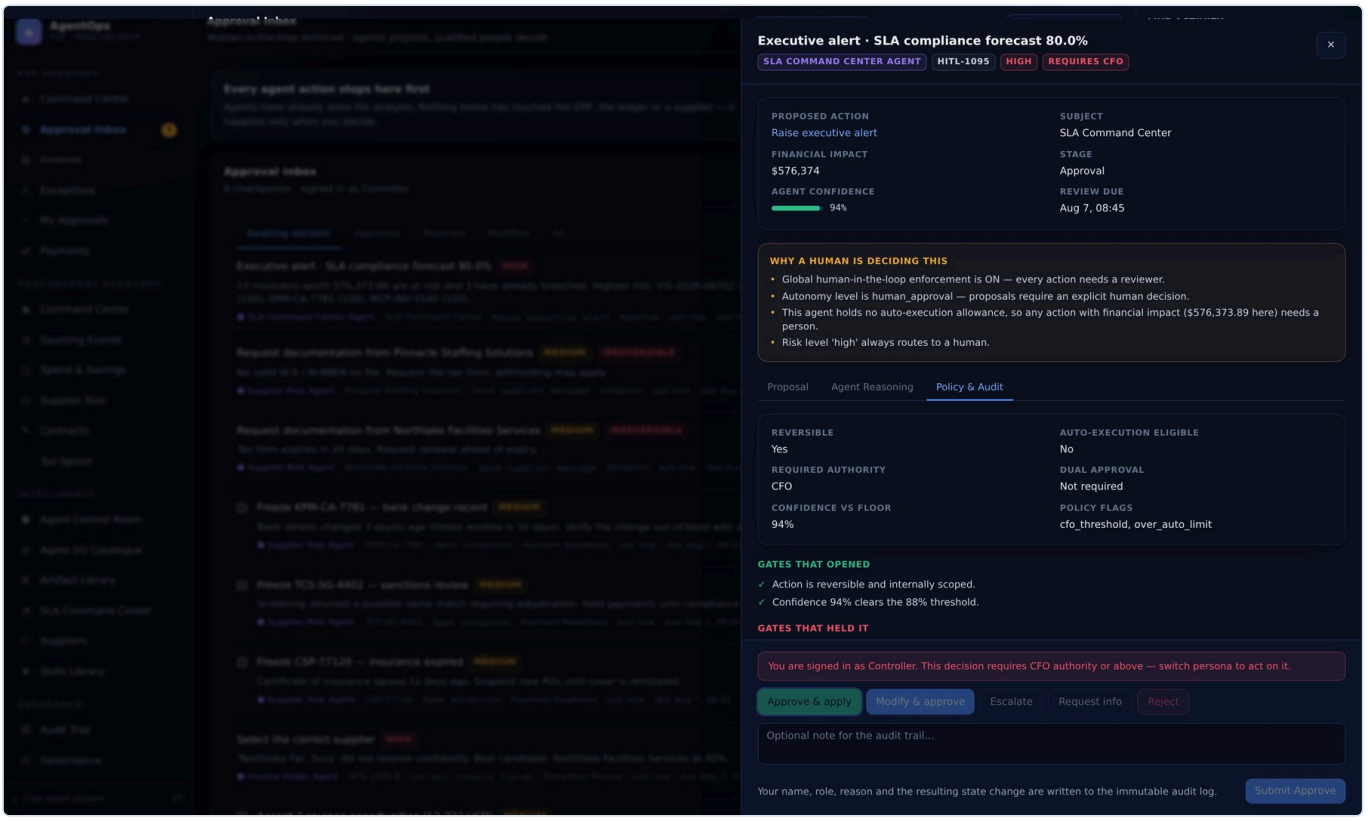
WHAT TO LOOK AT

1. **Persisted, not regenerated**
   Read from `agent_executions.plan` / `.observations` / `.evidence`. Re-opening this in a year shows the same thing.
2. **Decision rules are quantitative**
   e.g. "Amount variance +8.00% against a 3% tolerance" — a rule you can argue with.
3. **The narrator cannot change the outcome**
   The policy engine decided before the LLM was asked to explain it.

PART 02 · THE CHECKPOINT

Policy & audit — every gate, opened or blocked

All seven gates with their verdicts, the required role, and the audit entries this decision will write.



Seven gates, each with its verdict. `allow_auto_execute` is true only when every one opens, which is why in the default configuration this tab is a list of reasons the action stopped. The flags shown — `controller_threshold`, `cfo_threshold`, `dual_approval` — are the ones the engine actually set for this proposal, driven by its financial impact. Approving from here appends a row to `audit_logs` whose hash covers the previous row, which is what makes the history tamper-evident.

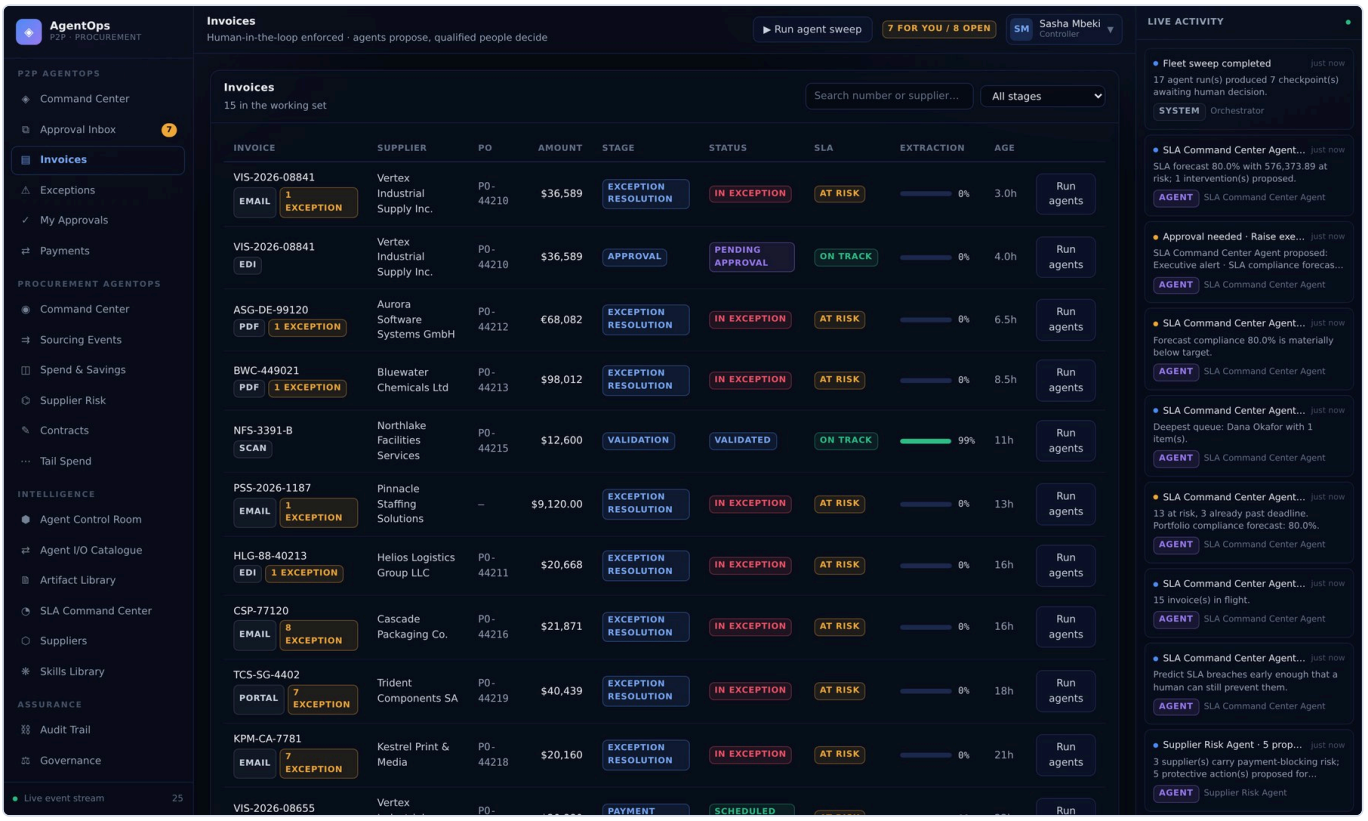
WHAT TO LOOK AT

1. **Default deny**
   `allow_auto_execute` needs *every* gate to open. Any single block sends it here.
2. **Value-based escalation is visible**
   The flags — `controller_threshold`, `cfo_threshold`, `dual_approval` — are the ones the engine actually set.
3. **The decision writes the chain**
   Approving appends a hash-chained `audit_logs` row covering the previous row's hash.

PART 02 · P2P AGENTOPS

Invoices — the working set

Every in-flight invoice with its stage, match result, owner and agent activity.



The working set. `stage` is the single most important column: the orchestrator maps a stage to exactly one owning agent, so an invoice in `matching` belongs to Three-Way Match and nothing else will act on it. That mapping is the whole state machine — there is no separate workflow engine to reason about. Filtering by stage is the fastest way to see one agent's caseload.

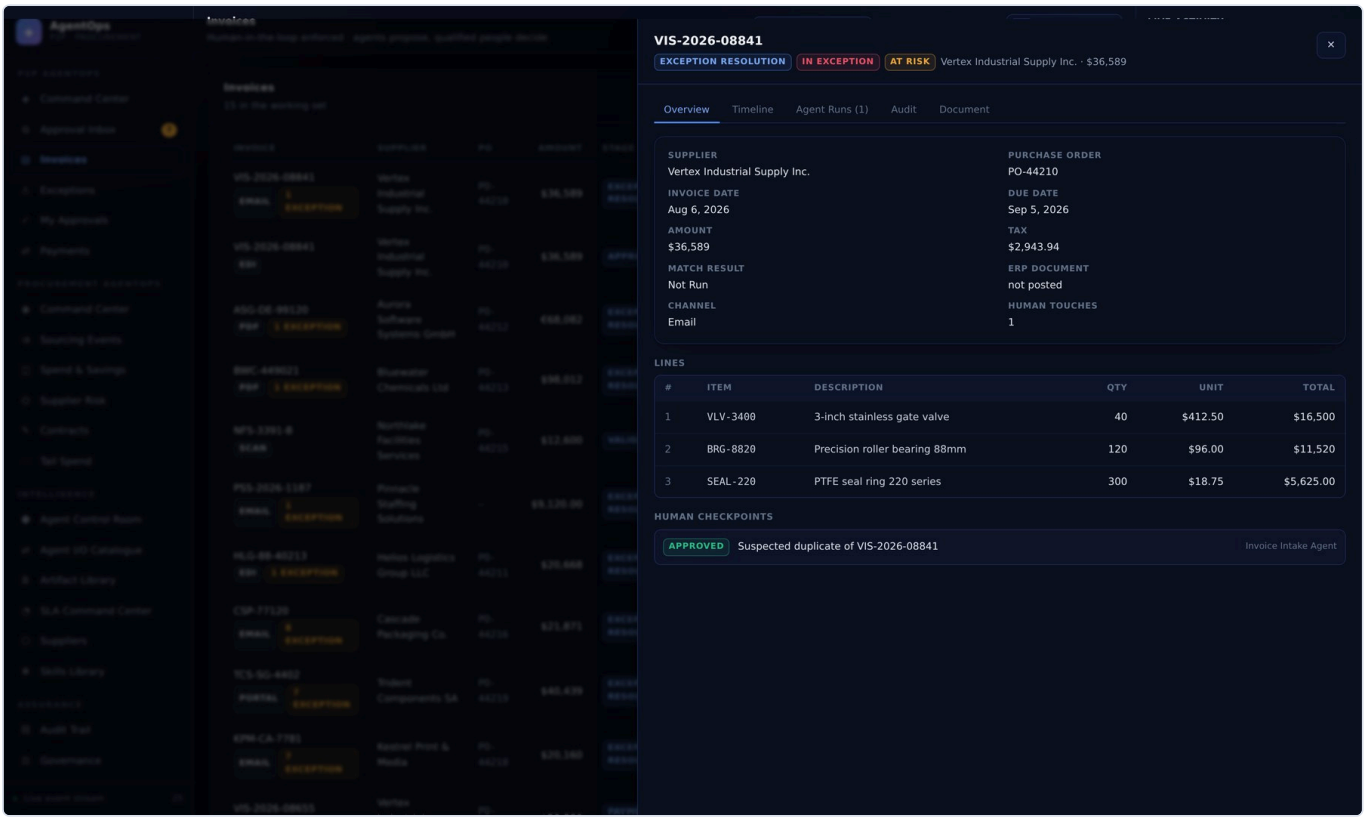
WHAT TO LOOK AT

1. **`stage` decides which agent owns it**
   The orchestrator maps stage → agent. `matching` belongs to Three-Way Match, and nothing else touches it.
2. **Match result is stored, not derived**
   The full line-level analysis is kept on `invoices.match_details` for audit.
3. **Filter by stage to follow one agent**
   The fastest way to see a single agent's working set.

PART 02 · P2P AGENTOPS

Invoice detail — timeline, runs and source document

Every event, every agent run and every human decision against one invoice, in order.



Everything that has happened to one invoice, in order: events, agent runs, human decisions, and the source document the extraction actually read. The timeline is a projection of `workflow_events`, which is append-only — nothing in this view can be edited, from the UI or otherwise. Each agent run links to the checkpoints it produced, and each checkpoint to the payload that was ultimately applied, so a reader can walk from "why is this invoice on hold" back to the observation that caused it.

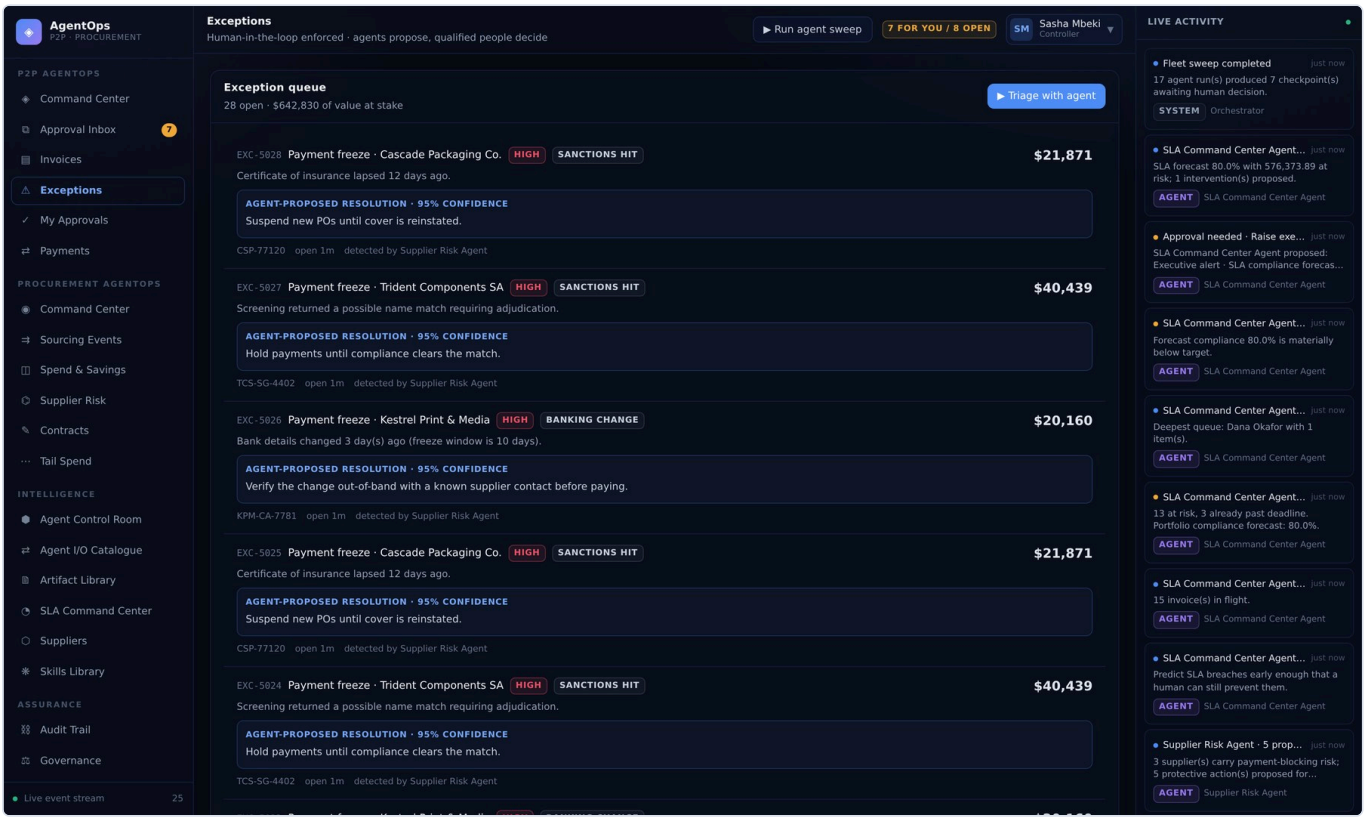
WHAT TO LOOK AT

1. **The timeline is `workflow_events`**
   Append-only and derived. Nothing in it can be edited from the UI.
2. **Agent runs link to their checkpoints**
   Run → proposal → decision → applied payload, all navigable.
3. **The source document is retained**
   Extraction is auditable against the text it actually read.

PART 02 · P2P AGENTOPS

Exceptions — the agent's diagnosis, priced

Open cases with the proposed resolution and the alternatives costed out.



An exception is a case, not an error. The agent diagnoses it, proposes a resolution, and prices the alternatives — and each alternative carries its own payload, so choosing one substitutes it into the checkpoint without the reviewer retyping anything. The confidence floor here is worth internalising: below 0.70 the agent proposes no resolution at all and asks for a human owner instead. A confidently wrong resolution on a disputed invoice costs more than no resolution.

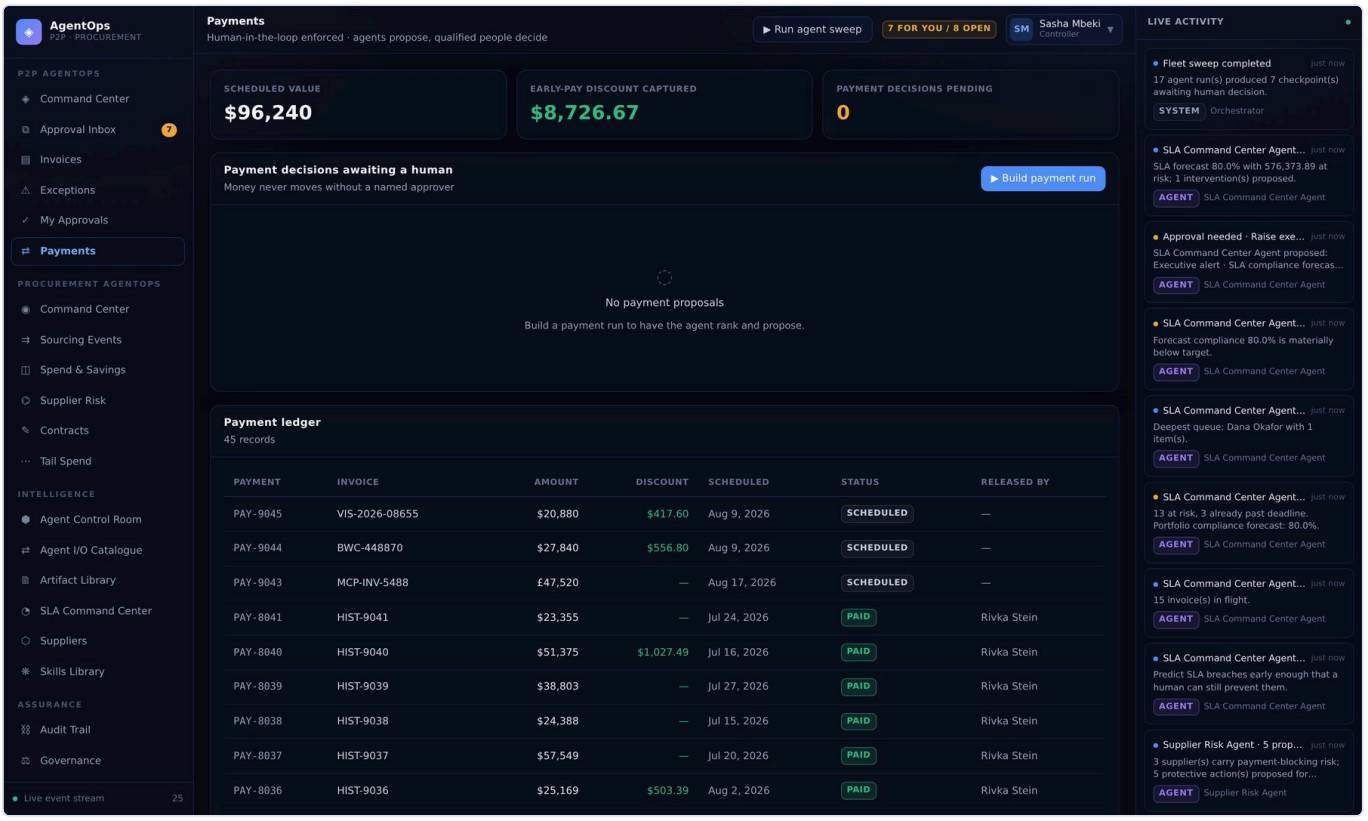
WHAT TO LOOK AT

1. **Alternatives carry payloads**
   Picking one substitutes its payload into the checkpoint — the reviewer never retypes anything.
2. **Below 70% confidence it proposes nothing**
   Exception Resolution asks for a human owner instead. A confident wrong answer is worse than none.
3. **Severity is computed from impact**
   Not a label the agent picked; a function of the money at stake.

PART 02 · P2P AGENTOPS

Payments — scheduling, discounts and release

Payment-run proposals ranked by priority score, with the early-pay discount at stake quoted.



Payment Readiness produces the run in lifecycle order — post to ERP, then release what is due, then hold what is blocked, then schedule the rest. The priority score is shown decomposed rather than as a single number, because a single number is not something a treasury analyst can review. One behaviour is easy to miss and matters: a payment whose supplier is on hold is never proposed at all. Proposing it and expecting the reviewer to catch it would be a trap.

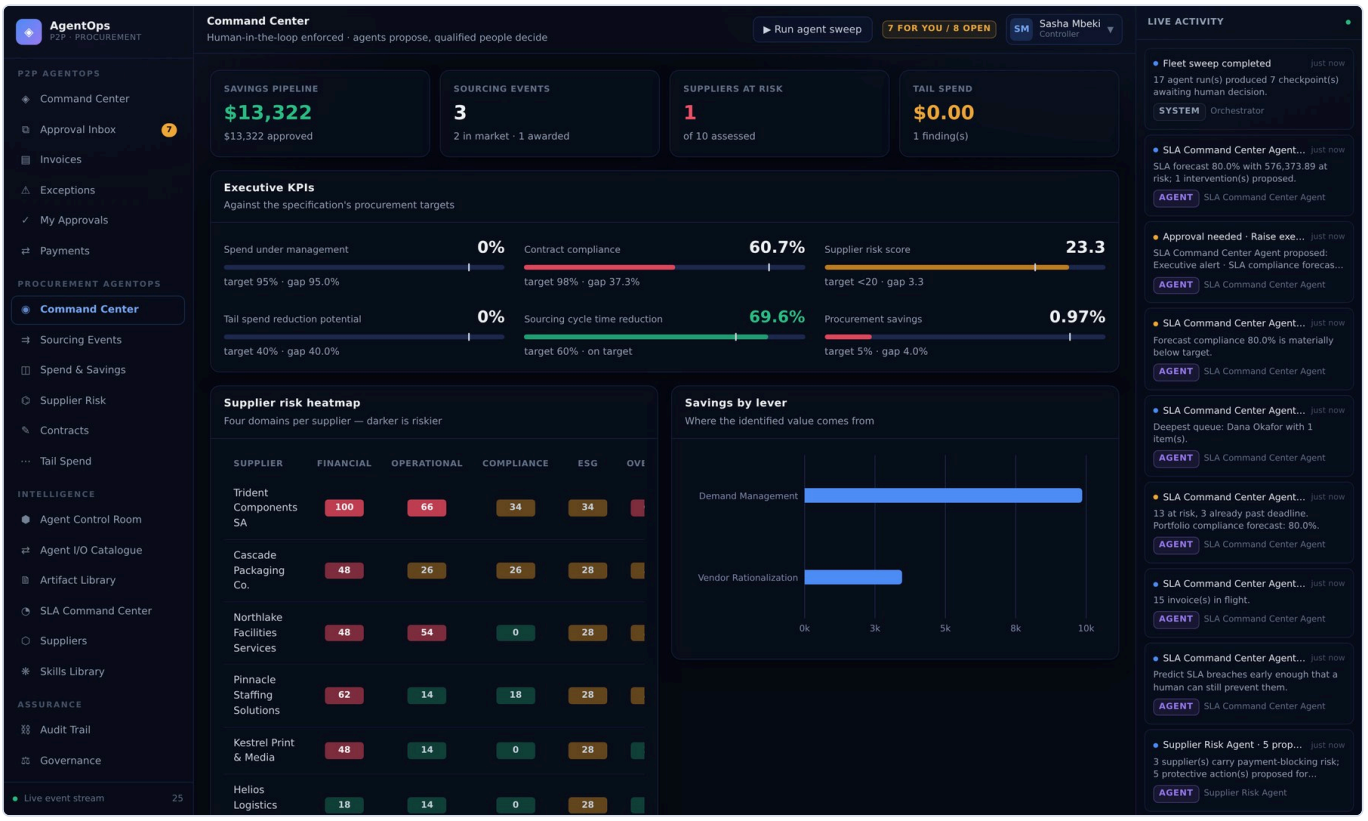
WHAT TO LOOK AT

1. `due_risk + supplier_tier + discount_value + sla_risk`
   The breakdown is shown, not just the total — a total on its own is not reviewable.
2. **Blocked suppliers never appear**
   Payment Readiness skips them entirely rather than proposing a release that would be rejected.
3. **Release is irreversible**
   Always a Controller, always a human, at any autonomy level.

PART 02 · PROCUREMENT AGENTOPS

Procurement Command Center — six executive KPIs

Spend under management, contract compliance, supplier risk, tail spend, cycle time and savings.



The strategic half of the platform, on the same control plane. Six KPIs against the specification's targets, and most of them start off target on purpose — the demo's story is that approving the agents' proposals is what closes the gaps. Publishing the executive brief is a CFO-level, irreversible action for the obvious reason: it reaches an audience that will act on it and cannot be recalled.

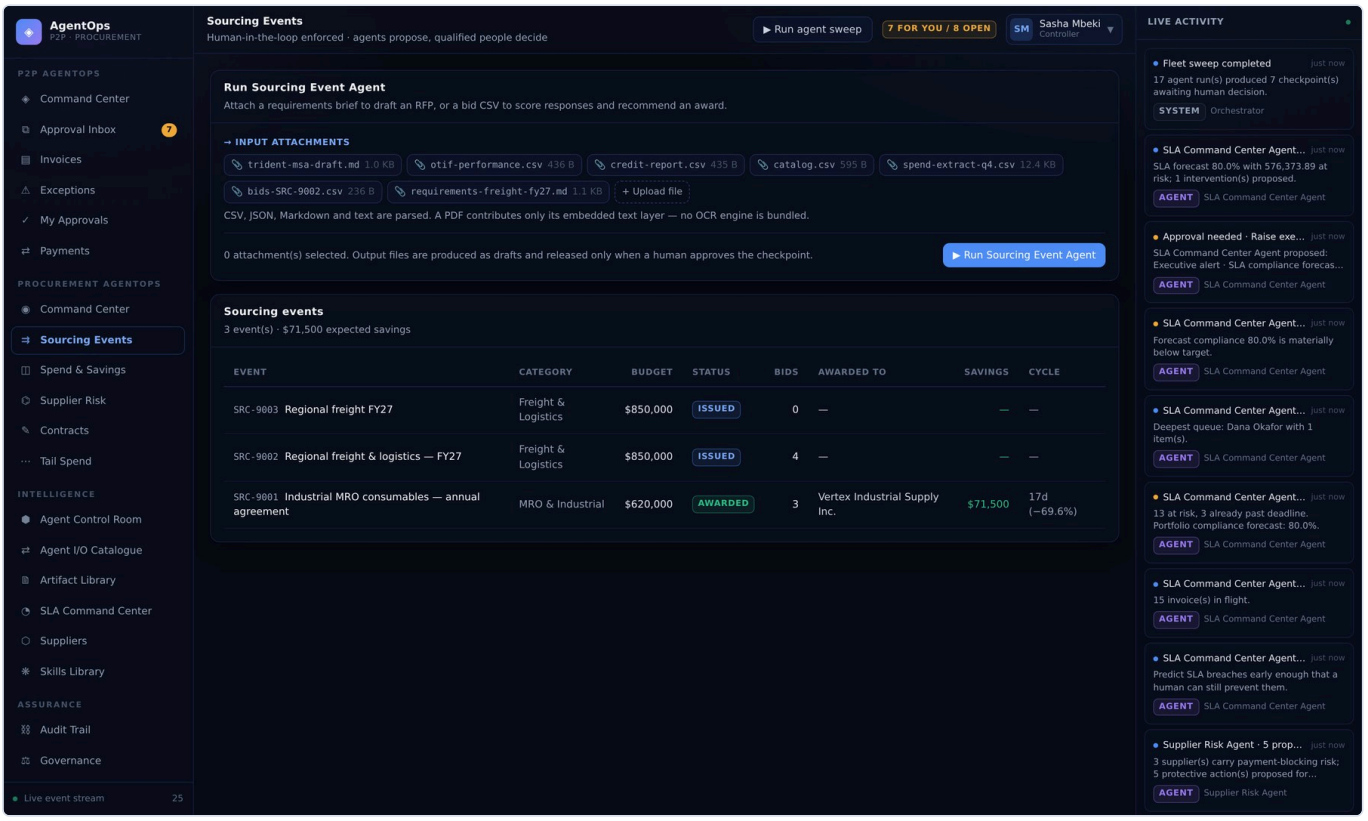
WHAT TO LOOK AT

1. **Most start off target on purpose**
   Approving the agents' proposals is what closes the gaps — that is the story worth showing.
2. **Publishing the brief is CFO-level**
   It reaches an audience that will act on it and cannot be unsent.
3. **Heatmap data is an artifact**
   Produced as a downloadable `.json`, born a draft like every other deliverable.

PART 02 · PROCUREMENT AGENTOPS

Sourcing Events — RFP to award

The RFP pipeline, bid scorecards and the award decision, with weights published up front.



Two design decisions here are worth arguing about, so they are made explicitly. First, compliance is a gate and not a weight — a supplier failing a mandatory rule is excluded from the shortlist rather than scored down, because a weighted score can always be outvoted by an attractive price. Second, issuing an RFP carries a financial impact of zero. It invites bids and commits nothing; the award is where money moves, so that is where value-based escalation and dual approval apply.

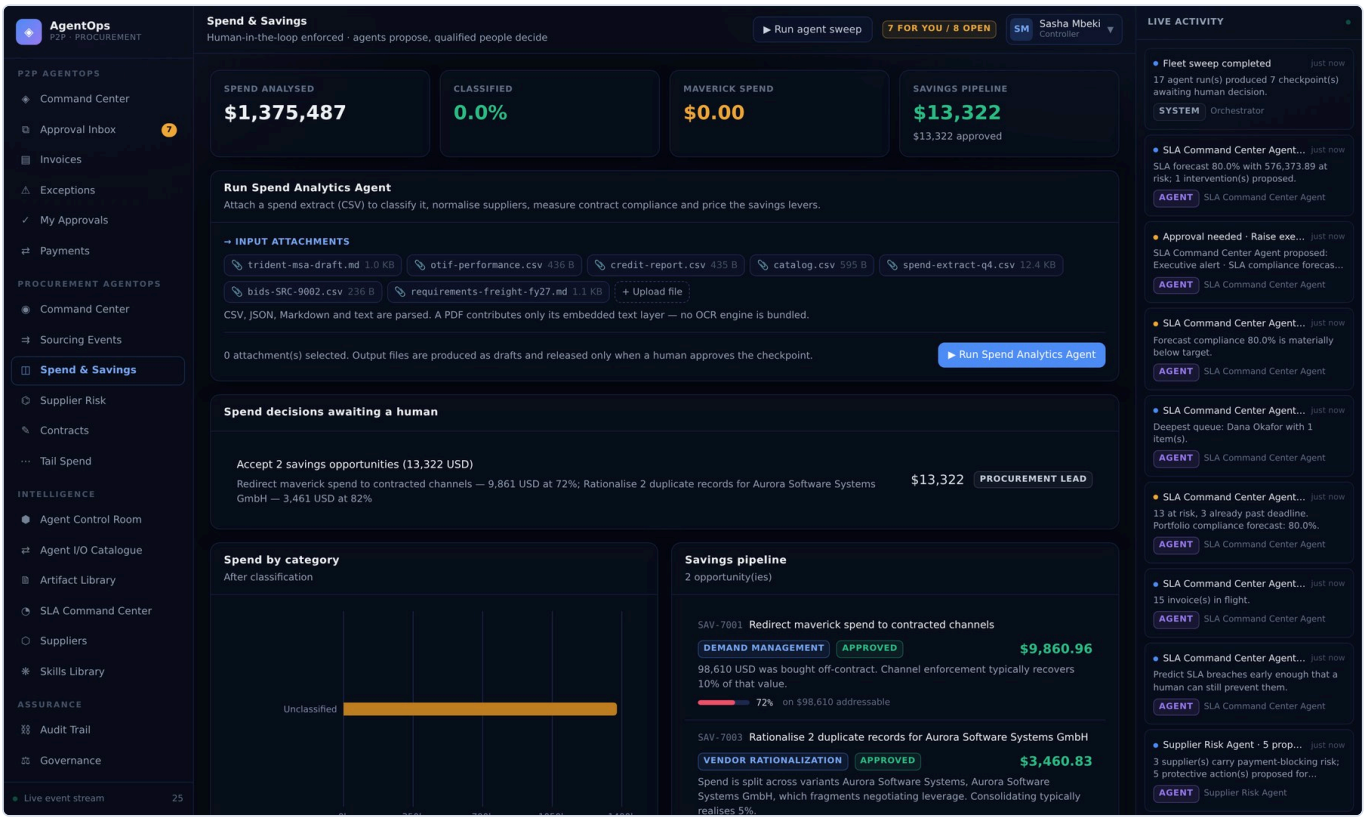
WHAT TO LOOK AT

1. **Compliance is a gate, not a weight**
   A supplier failing a mandatory rule is excluded, not scored down. A weight can be outvoted by price; a gate cannot.
2. **Issuing an RFP has zero financial impact**
   It invites bids and commits nothing. The award is where the money moves — and where CFO escalation applies.
3. **Above $100k the award needs two approvers**
   Compared by `user.id`, so one person signing twice is one approval.

PART 02 · PROCUREMENT AGENTOPS

Spend & Savings — classification and the savings pipeline

Classified spend by category, coverage, and savings priced per lever.



Classification, savings and — importantly — the residual. Shipping the unclassified remainder as its own downloadable file is the point: a coverage percentage you cannot look inside is not something a category manager can act on. Savings are modelled per lever at published rates rather than estimated per line, so two runs over the same extract produce the same number and the method can be argued with.

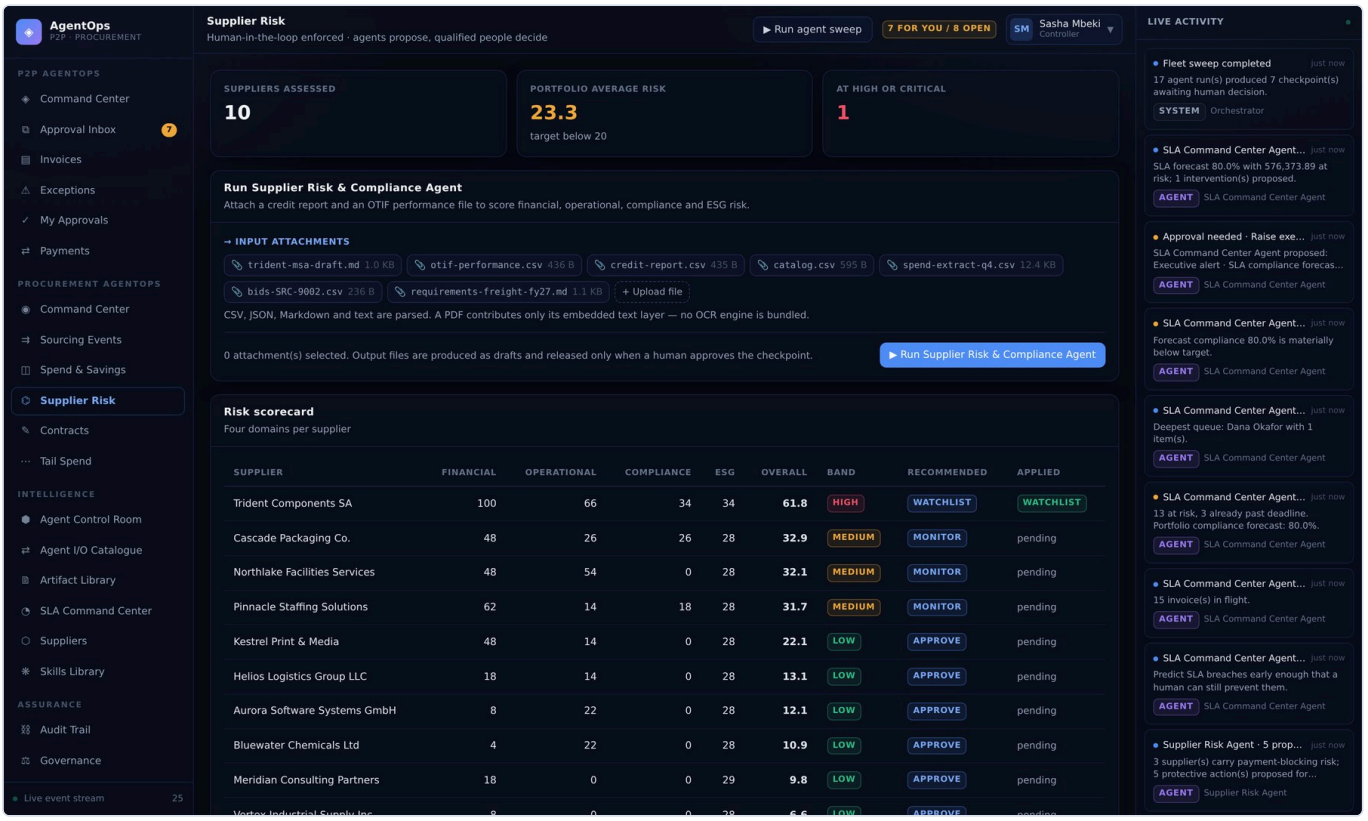
WHAT TO LOOK AT

1. **The unclassified residual ships as a file**
   A coverage percentage you cannot see inside cannot be acted on.
2. **Savings are modelled, not estimated**
   Published rates per lever, so two runs over the same data agree and the method is auditable.
3. **Below 85% coverage it escalates**
   The savings numbers would rest on too little classified spend.

PART 02 · PROCUREMENT AGENTOPS

Supplier Risk — four domains, one disposition

Financial, operational, compliance and ESG risk scored separately, then blended.



Four domains scored separately, then blended 0.30 / 0.25 / 0.30 / 0.15. The blend prevents one bad domain from condemning a supplier by itself, and carrying each domain through to the scorecard prevents the blend from hiding which one drove the result. The recommendation is written to `risk_assessments`; `applied_disposition` stays null until a human approves, which is why the recommendation and the applied outcome are separate columns.

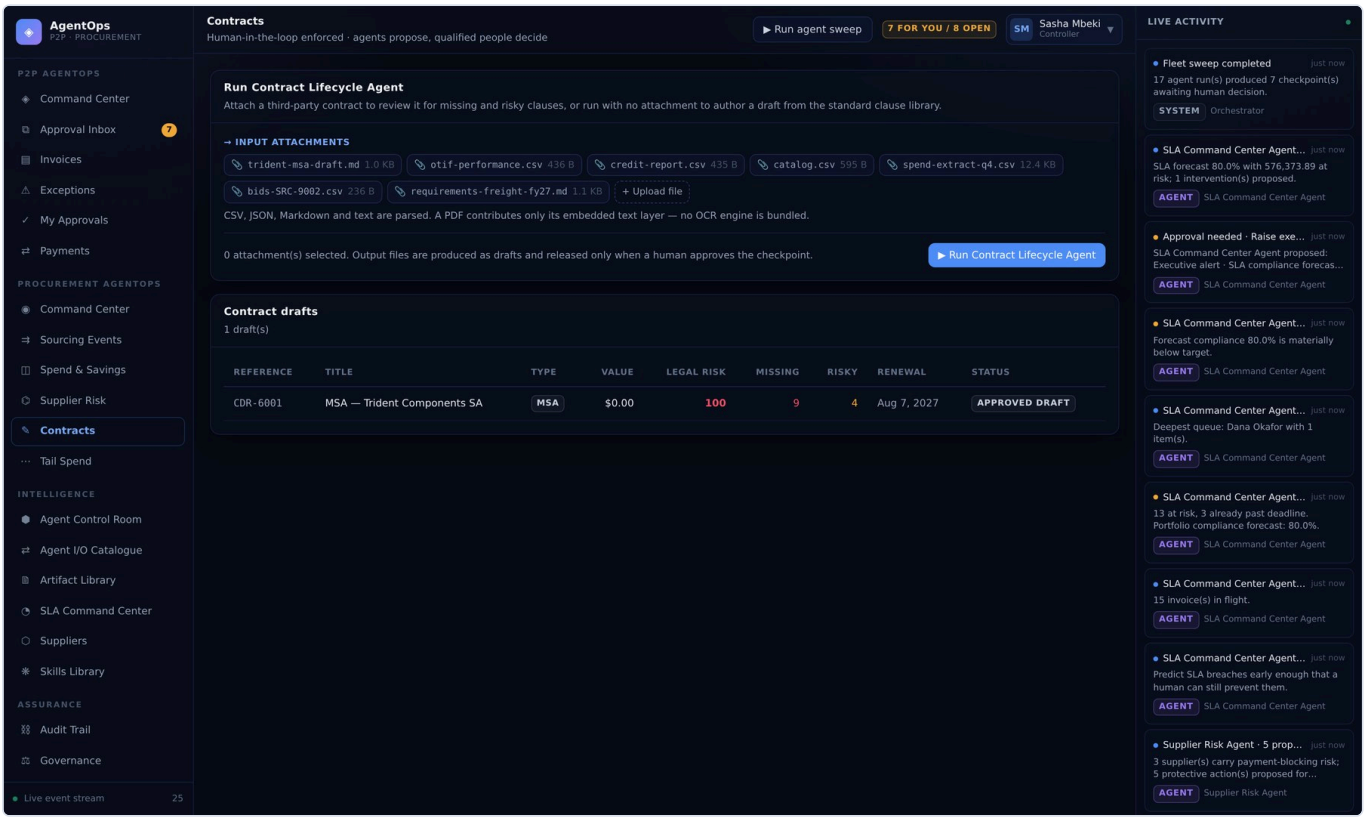
WHAT TO LOOK AT

1. **0.30 / 0.25 / 0.30 / 0.15**
   A weighted blend, so one bad domain does not by itself condemn a supplier — but each domain is carried through.
2. `applied_disposition` **stays null**
   The assessment records the recommendation. The vendor master changes only on approval.
3. **Thin evidence lowers confidence**
   Two or more "no data" findings drop it to 0.72 rather than scoring confidently on nothing.

PART 02 · PROCUREMENT AGENTOPS

Contracts — clause review that withholds signature

Drafts, clause findings, obligations and renewals, with the legal risk score built additively.



The clause review is the most opinionated agent in the platform, and deliberately so: while mandatory clauses are missing or risky ones are present, the "issue for signature" proposal is *never made*. A reviewer cannot approve a checkpoint that does not exist. The sample MSA shipped with the demo contains unlimited liability, a unilateral price-change clause, a buyer-indemnifies-supplier clause and a silent auto-renewal, so the analysis has something real to find — and it scores badly enough that signature is withheld.

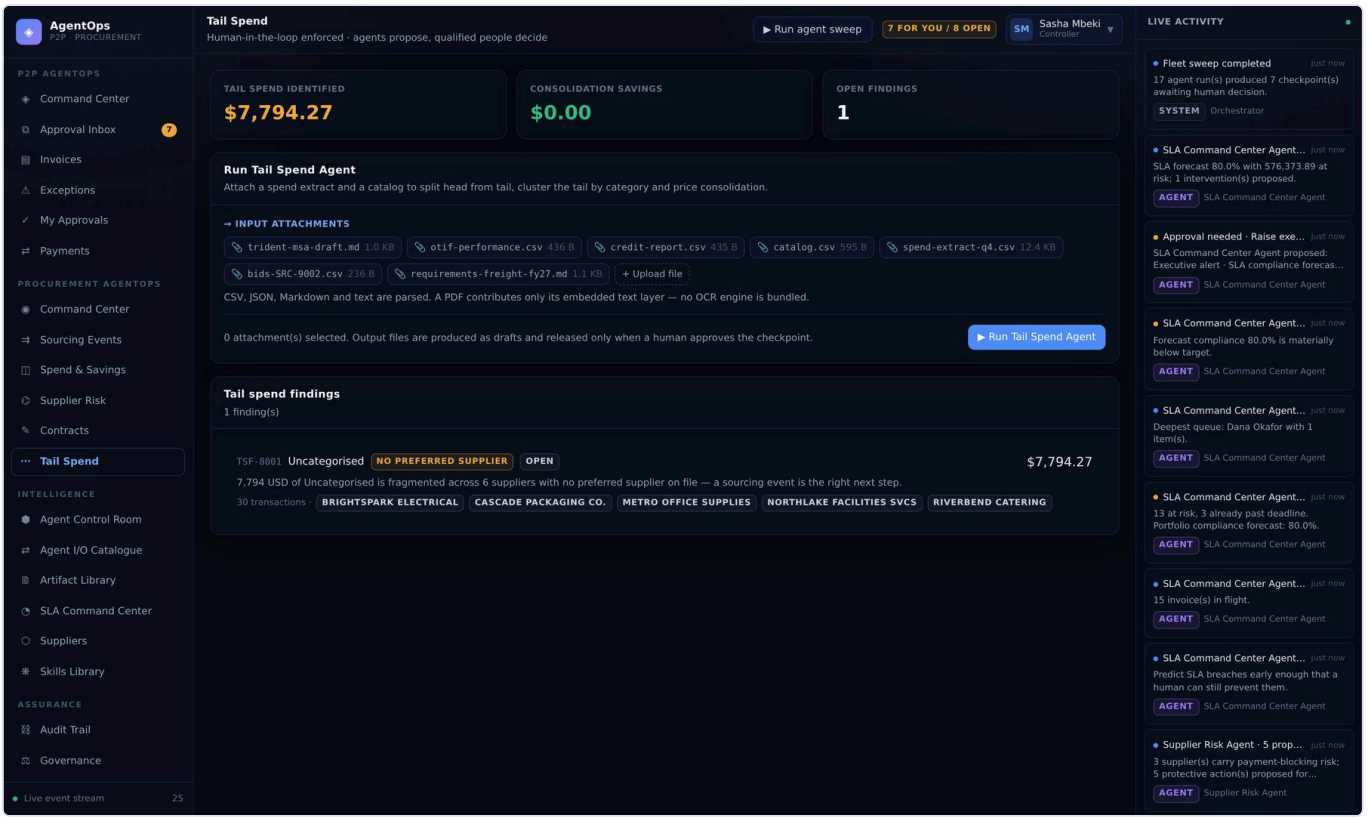
WHAT TO LOOK AT

1. **Signature is not offered on bad paper**
   While mandatory clauses are missing or risky ones present, the proposal is never made — so it cannot be approved.
2. **A missing clause carries weight too**
   An omission is a risk, not a neutral absence.
3. **Silent auto-renewal is flagged with its notice period**
   The risk is the date passing unnoticed, not the clause existing.

PART 02 · PROCUREMENT AGENTOPS

Tail Spend — the long tail, governed

Head/tail split on a Pareto cut, consolidation clusters and off-catalogue enforcement.



A Pareto cut splits head from tail, then the tail is clustered by category. Two floors keep the output actionable: a cluster must be worth at least $4,000 and contain at least two suppliers. Without the materiality floor the screen fills with sub-thousand-dollar clusters that cost more to action than they save. Where no preferred supplier exists the finding is recorded with zero modelled savings rather than a number nobody can realise.

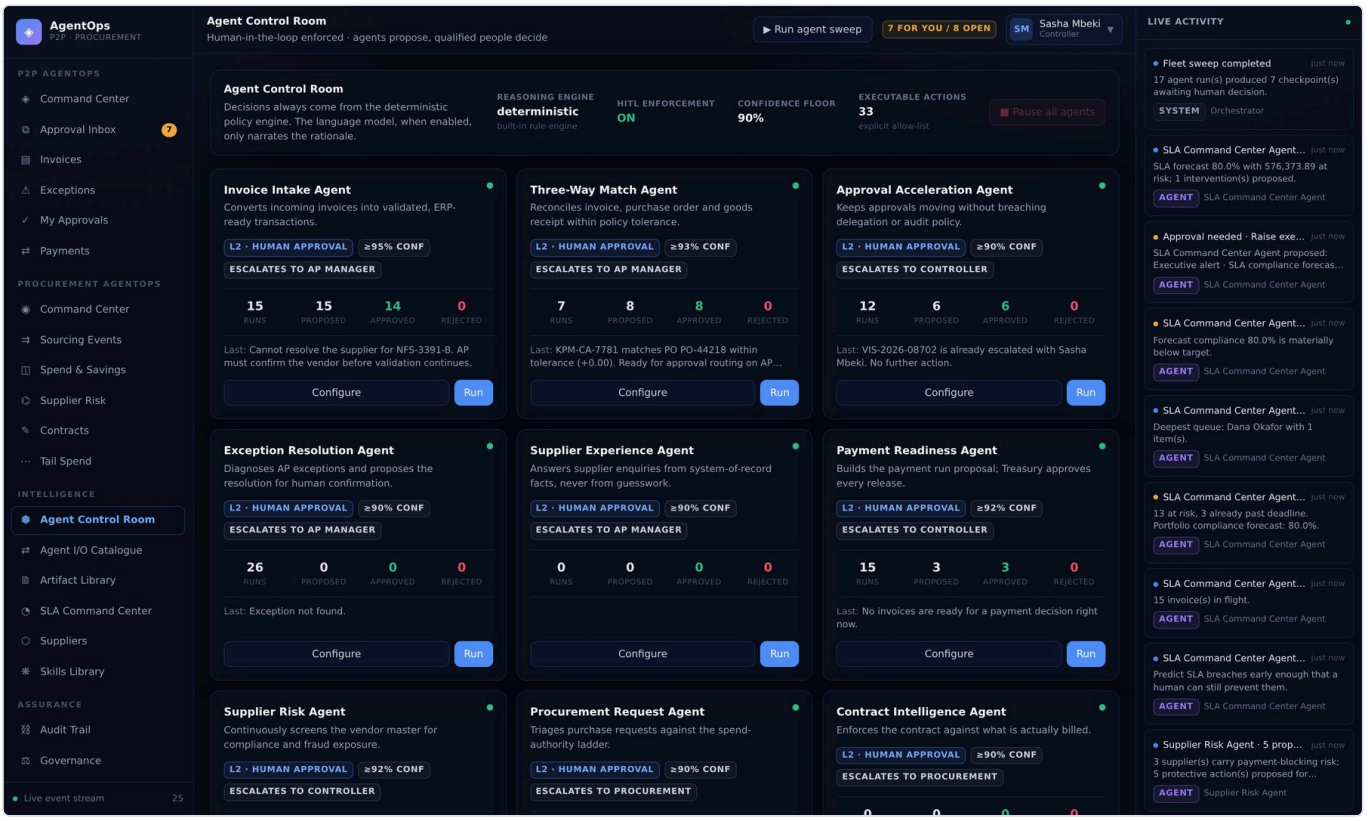
WHAT TO LOOK AT

1. **$4,000 materiality floor**
   Without it, clustering surfaces dozens of clusters that cost more to action than they save.
2. **No preferred supplier → zero savings claimed**
   The finding is recorded honestly rather than claiming savings nobody can realise.
3. **Enforcement needs a substitute**
   Where no catalogue item exists, the buyer had no compliant option — so nothing is proposed.

PART 02 · INTELLIGENCE

Agent Control Room — governance per agent

Autonomy level, confidence threshold, financial ceiling, permitted actions and run history.



Per-agent governance, and the fastest development loop in the platform: change a `decide()` branch, click Run, read the resulting checkpoint. All sixteen agents ship at L2 — every proposal waits for a human — and L4 is disabled outright while global enforcement is on. The fleet kill switch is here too; a disabled agent returns `blocked_by_policy` without executing its plan.

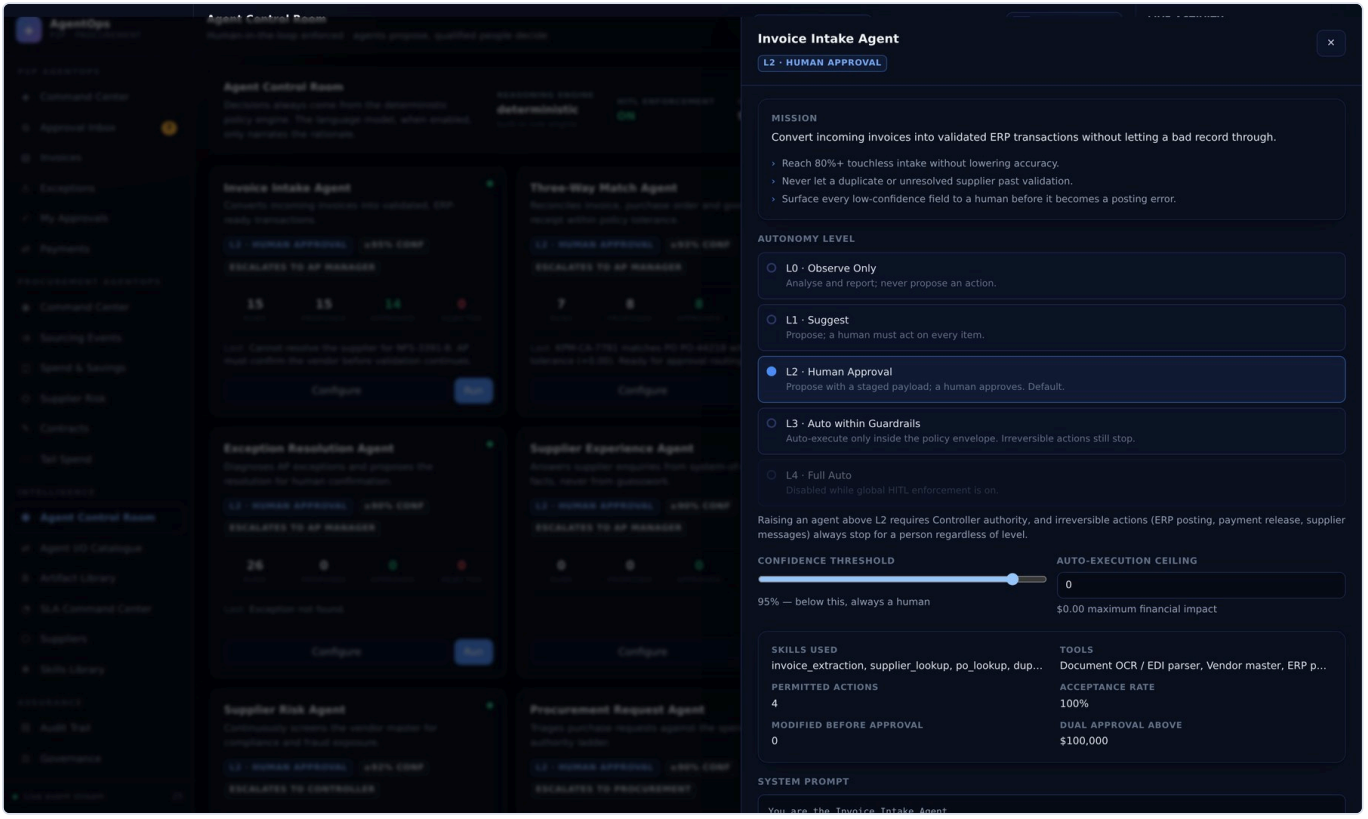
WHAT TO LOOK AT

1. **All 16 ship at L2 · Human Approval**
   Every proposal waits. L4 is disabled entirely while global enforcement is on.
2. **The fleet kill switch**
   `P2P_AGENTS_PAUSED` or the button — a disabled agent returns `blocked_by_policy` without running.
3. **Run one agent on demand**
   The fastest development loop: change `decide()`, click run, read the checkpoint.

PART 02 · INTELLIGENCE

Agent configuration — the envelope you can move

Everything the policy engine reads about this agent, editable and audited.



These fields are rows in `agent_configs`, seeded from the class defaults and then owned by the database. That is the important part: governance is data, so tightening a confidence threshold or lowering a ceiling is an audited edit rather than a deploy. Raising autonomy does not bypass human review — the global switch is gate 1, and gates 2 through 7 apply regardless of where it is set.

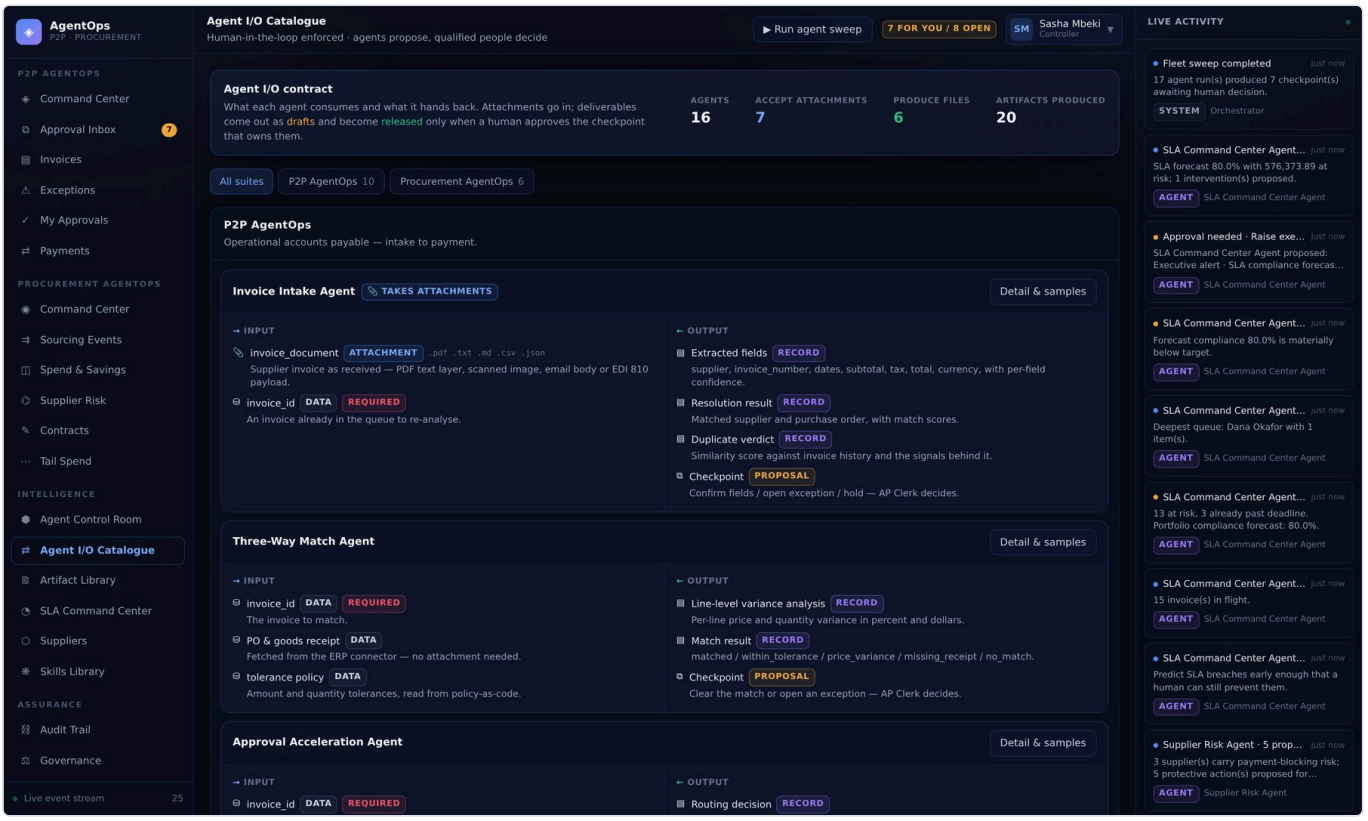
WHAT TO LOOK AT

1. **These are `agent_configs` rows**
   Seeded from the class defaults, then owned by the database. Governance is data.
2. **Raising autonomy does not bypass HITL**
   Gate 1 is the global switch; gates 2–7 still apply either way.
3. **The prompt is shown, not hidden**
   Generated by `BaseAgent.prompt_template()` from the declared identity.

PART 02 · INTELLIGENCE

Agent I/O Catalogue — the contract, all 16 agents

What each agent consumes and what it hands back, with formats and the files it has produced.



The contract for all sixteen agents on one screen, rendered from the `IOSpec` objects each agent declares in code. There are five kinds — `attachment`, `data`, `artifact`, `record`, `proposal` — and the last is always a checkpoint, which is how you can tell at a glance what each agent will ask a human to decide. Files the agent has actually produced are listed underneath, downloadable, with their draft or released state visible.

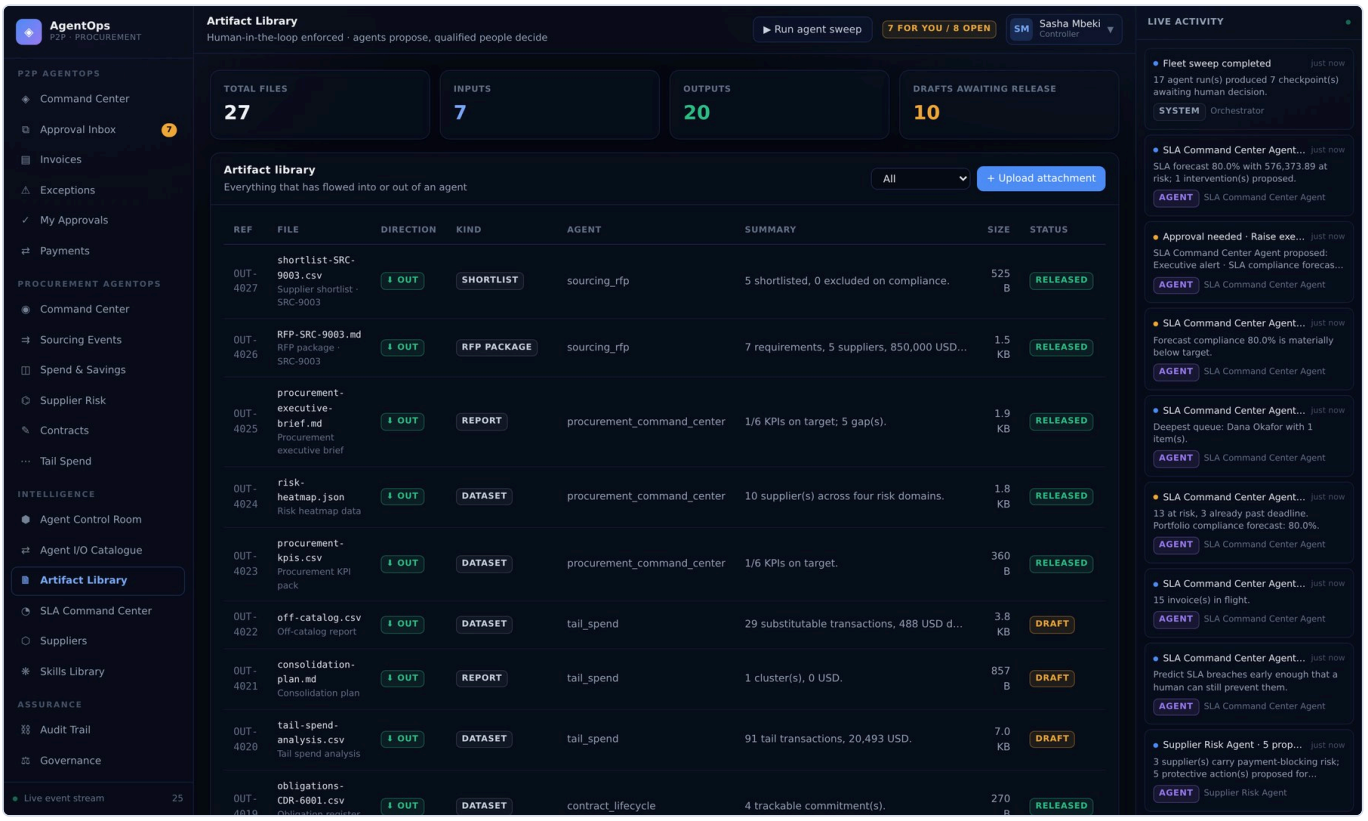
WHAT TO LOOK AT

1. **Rendered from `IOSpec` declarations**
   Not a written document — the same objects the agent declares in code.
2. **Five kinds**
   `attachment`, `data`, `artifact`, `record`, `proposal` — the last one is always a checkpoint.
3. **Produced files are downloadable**
   Straight from `artifacts`, with their draft/released state visible.

PART 02 · INTELLIGENCE

Artifact Library — documents in and out

One table, two directions. Uploaded attachments and agent deliverables side by side.



One table, two directions. Uploads arrive as `direction=input`; deliverables are written as `direction=output` with `status=draft`. A draft becomes released only when the checkpoint that owns it is approved, and rejecting that checkpoint leaves the files as drafts — evidence of what was considered, with nothing issued. Seven sample attachments ship seeded so an agent can be run against real input immediately.

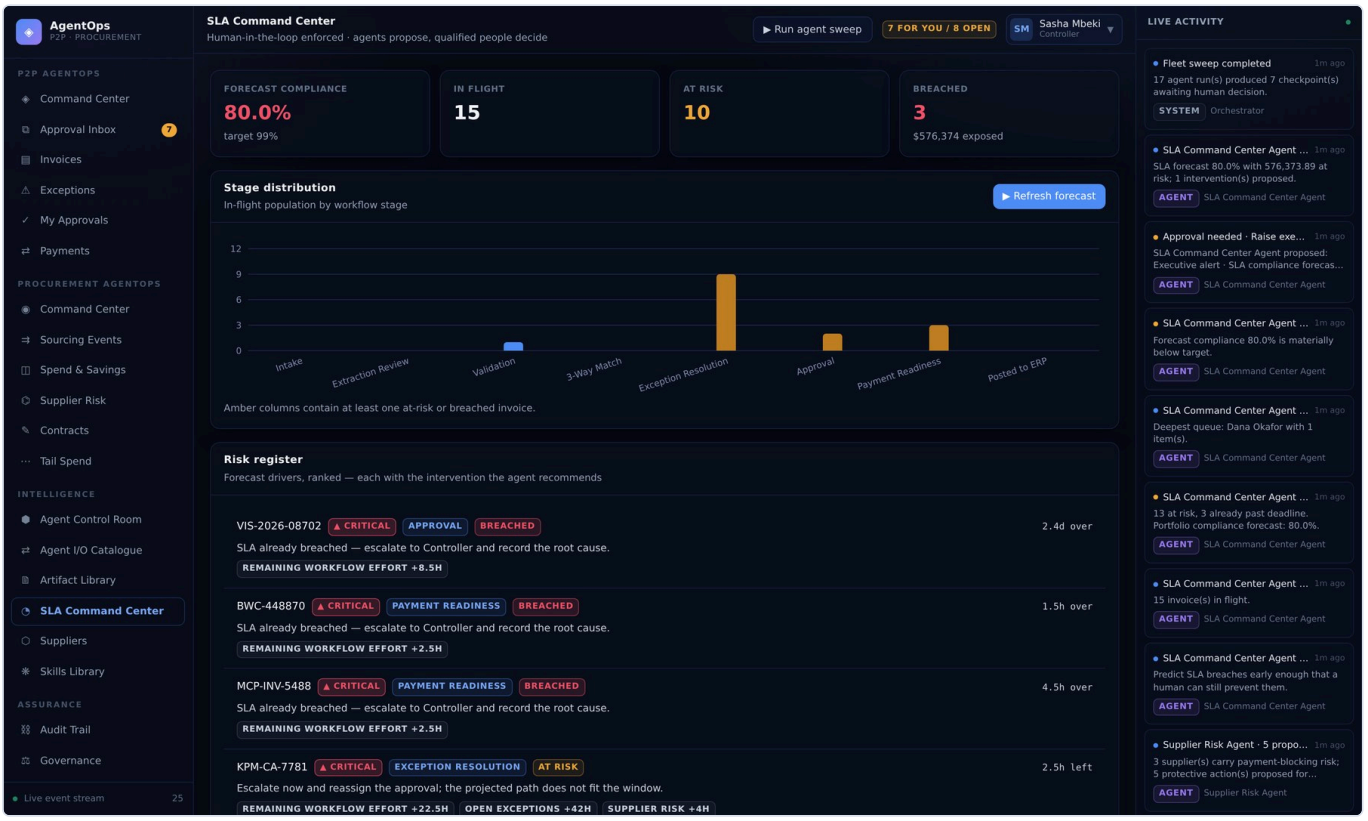
WHAT TO LOOK AT

1. **Outputs are born `draft`**
   They become `released` only when the checkpoint that owns them is approved.
2. **Rejection leaves them as drafts**
   Evidence of what was considered, with nothing issued.
3. **Seven samples ship seeded**
   So a demo can run an agent against real input with no preparation.

PART 02 · INTELLIGENCE

SLA Command Center — forecast, not report

Breach forecast per invoice with its drivers, and the interventions that would protect it.



A forecast, not a report. The prediction costs the stages an invoice has left rather than the whole workflow, and it excludes exception handling unless the invoice is actually in exception — charging every invoice for the worst case forecasts the entire portfolio at risk, which is both wrong and useless. Rebalancing proposals only move work to an approver whose limit covers the invoice; a move that creates an unapprovable assignment has relocated the problem.

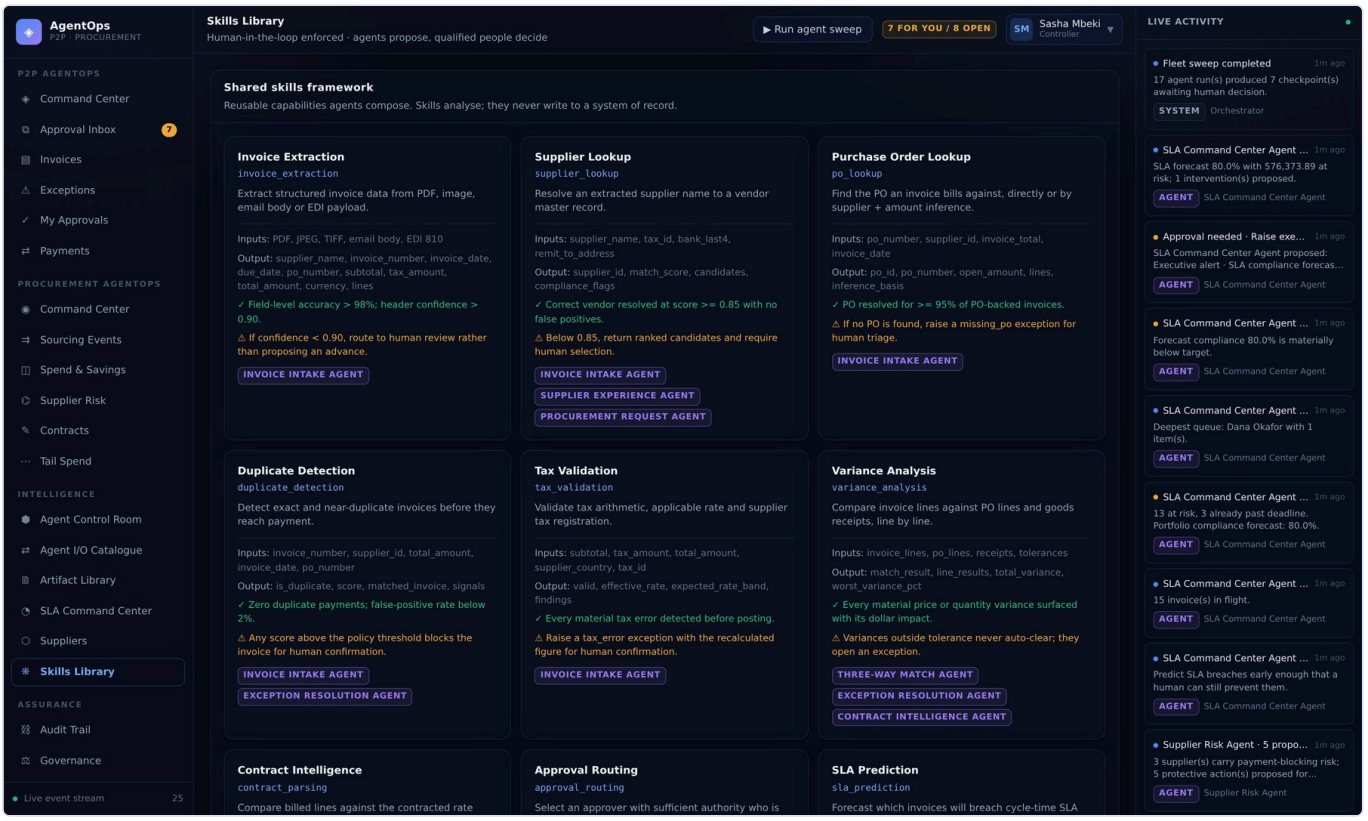
WHAT TO LOOK AT

1. **Costs remaining stages only**
   Charging every invoice for exception handling forecasts the whole portfolio at risk — wrong and useless.
2. **Rebalancing respects approval limits**
   A move that creates an unapprovable assignment has relocated the problem, not solved it.
3. **Below 95% forecast compliance it escalates**
   That is an executive condition, not a queue item.

PART 02 · INTELLIGENCE

Skills Library — 28 shared capabilities

The analysis layer the agents compose. A skill returns evidence; it never writes.



Twenty-eight analysis capabilities that the agents compose. A skill takes plain data and returns plain data — no session, no writes — which is why they are trivial to unit test and why the same `variance_analysis` serves Three-Way Match, Exception Resolution and Contract Intelligence. Their contracts in `skills//SKILL.md` are generated from the live descriptors, so the documentation cannot drift from the implementation.

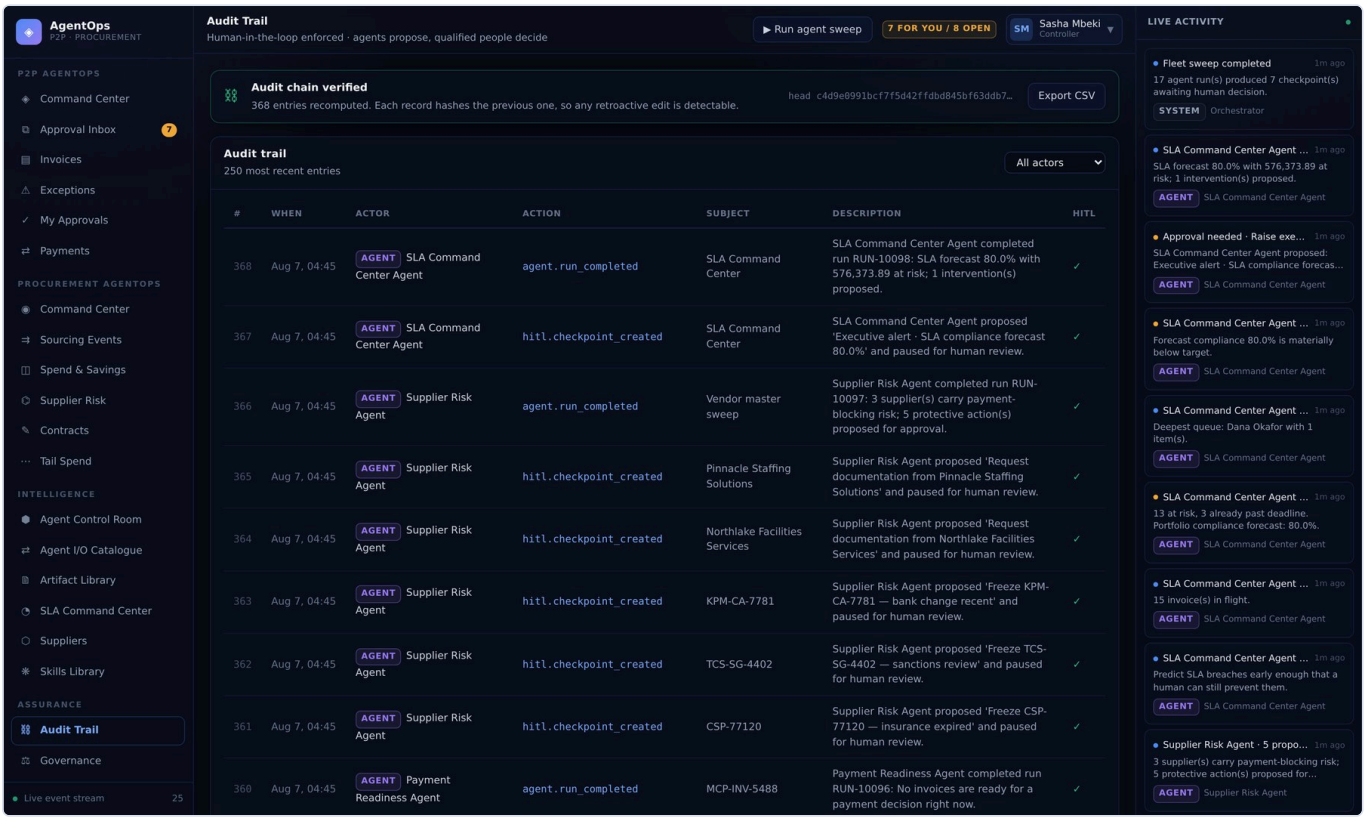
WHAT TO LOOK AT

1. **Contracts are generated**
   `skills/<name>/SKILL.md` from the live descriptors, so documentation cannot drift.
2. **Skills are pure functions of their inputs**
   No session, no writes — which is why they are trivially testable.
3. **Reused across agents**
   `variance_analysis` serves Three-Way Match, Exception Resolution and Contract Intelligence.

PART 02 · ASSURANCE

Audit Trail — hash-chained and verifiable

Every decision, who made it, on what evidence, and whether enforcement was on at the time.



Every decision, human or agent, with the actor, the entity, the before and after state, the confidence, and whether enforcement was on at the time. Each row stores `sha256(previous_hash ‖ canonical_payload)`, so the log is a chain rather than a table. `GET /api/audit/verify` replays it from genesis and names the first row that fails — which is what makes an out-of-band edit detectable rather than merely discouraged.

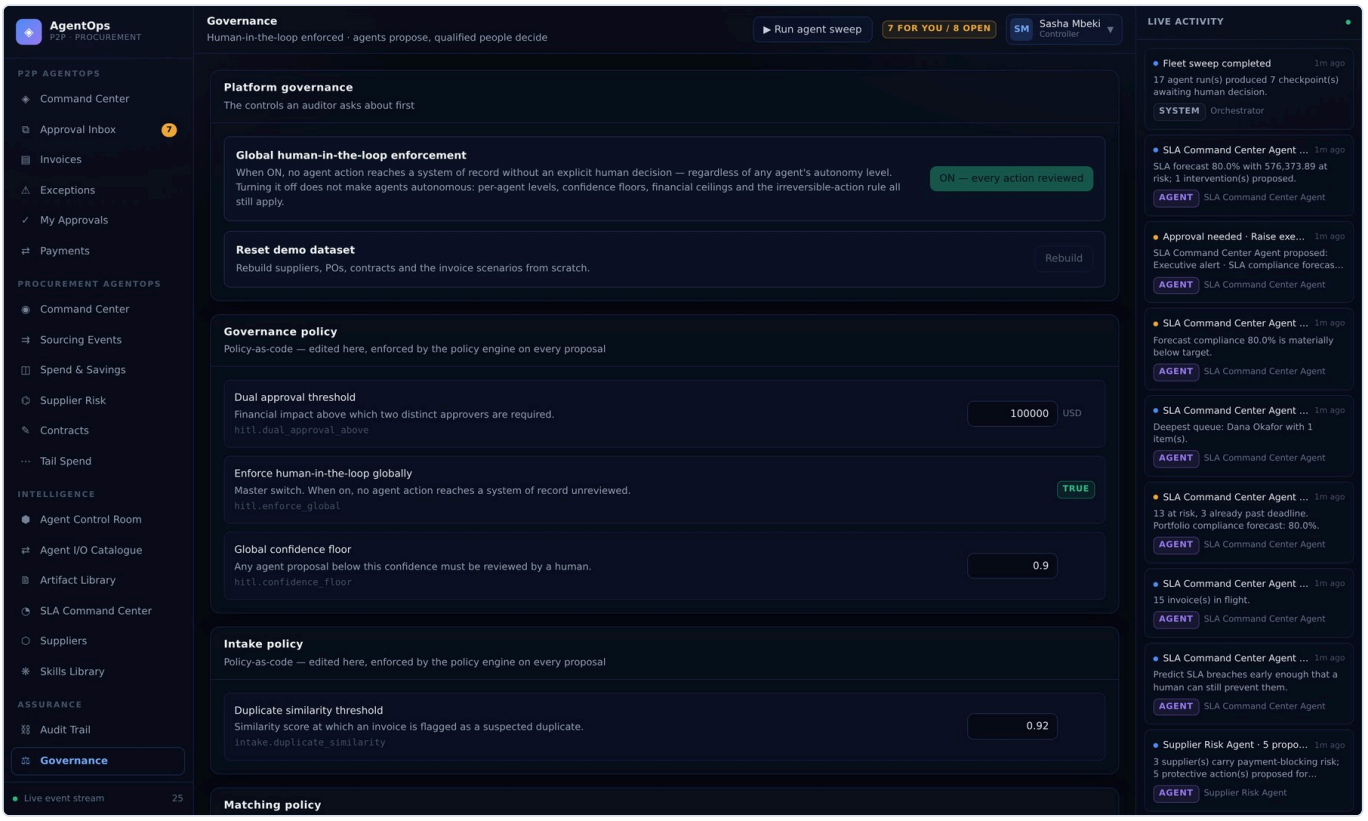
WHAT TO LOOK AT

1. `sha256(previous_hash ‖ payload)`
   Each row covers the one before it. Editing history breaks verification at that row.
2. `GET /api/audit/verify` **replays from genesis**
   And reports the first divergence, if any.
3. **Agent and human entries are distinguished**
   `actor_type` separates what the agent concluded from what a person decided.

PART 02 · ASSURANCE

Governance — policy as data

Fifteen rules, the master HITL switch, and the thresholds every agent reads on every run.



Fifteen rules, editable, audited, and read on every agent run. Loosening a match tolerance here changes agent behaviour on the next run with no deploy and a record of who changed it. The master switch is the one people ask about: turning it off opens gate 1 only. An L2 agent still stops, an irreversible action still stops, a low-confidence proposal still stops. It exists so the governance story can be demonstrated end to end, not as a bypass.

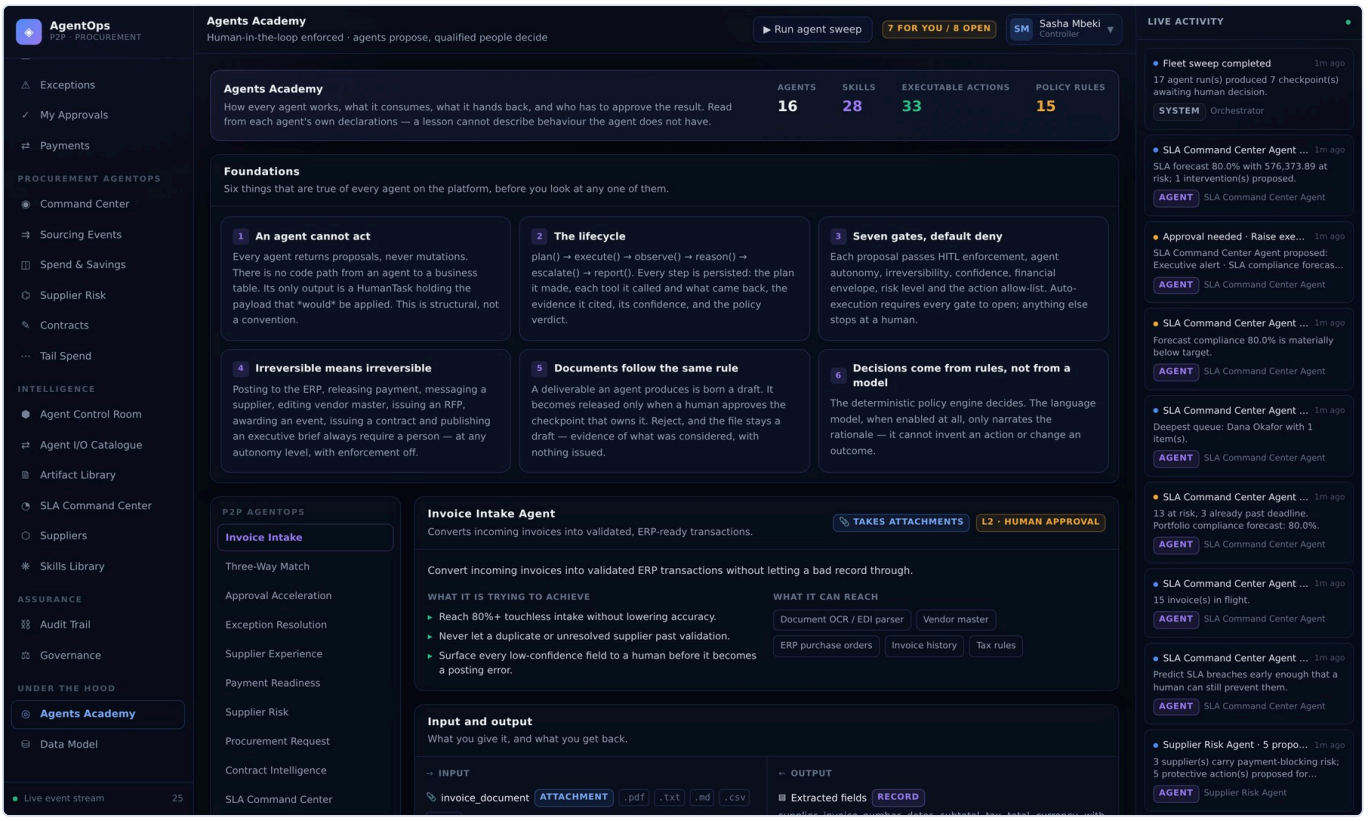
WHAT TO LOOK AT

1. **Changing governance is a data change**
   Tolerances and thresholds are rows in `policy_rules`, not constants in code.
2. **The master switch opens gate 1 only**
   An L2 agent still stops; an irreversible action still stops; a low-confidence proposal still stops.
3. **Every edit is audited**
   Who loosened the tolerance, when, and from what to what.

PART 02 · UNDER THE HOOD

Agents Academy — six foundations, then each agent

What is true of every agent before you look at any one of them.



The onboarding path for a developer with no context. Six foundations that hold for every agent, then a curriculum per agent — all of it read from the agents' own declarations, so a lesson cannot describe behaviour an agent does not have. Read the six foundations, then jump straight to the agent you are about to change.

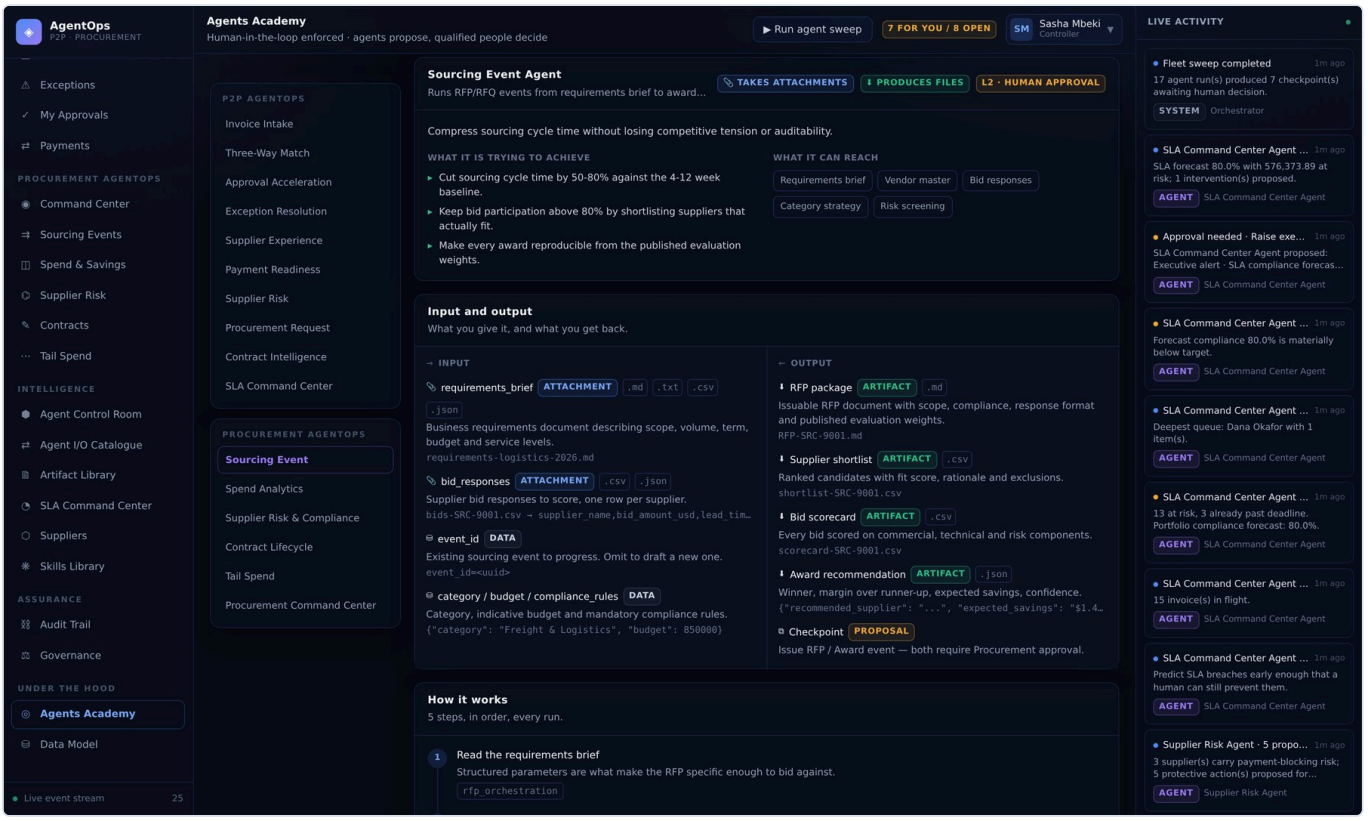
WHAT TO LOOK AT

1. **Read from the agents' own declarations**
   A lesson cannot describe behaviour the agent does not have.
2. **The foundations are the mental model**
   An agent cannot act · the lifecycle · seven gates · irreversible means irreversible · documents follow the same rule · rules decide, not a model.
3. **This is the onboarding path**
   New developer, no context: read the six, then pick the agent you are changing.

PART 02 · UNDER THE HOOD

Academy — input and output, per agent

The question the screen exists to answer: what do I give it, and what do I get back?



The question this screen exists to answer: what do I give this agent, and what do I get back? Each field carries its kind, accepted formats and a worked example — enough to construct a call without opening the source. Attachments arrive already parsed: CSV as typed rows with a field list, JSON as records, Markdown as text, PDF as its embedded text layer. An agent never touches storage.

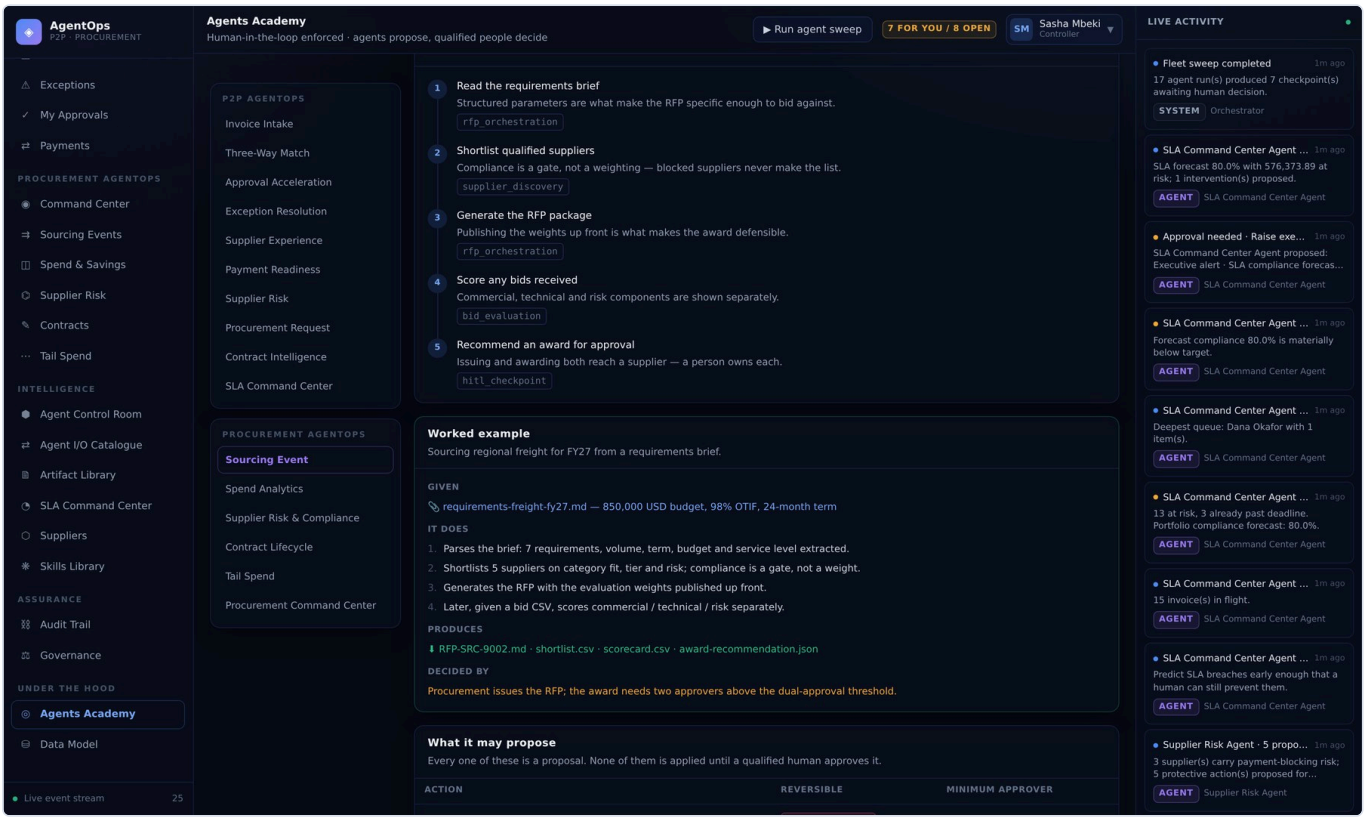
WHAT TO LOOK AT

1. **Kind, format and example per field**
   Enough to construct a call without reading the source.
2. **Attachments arrive parsed**
   CSV as typed rows, JSON as records, PDF as its text layer — the agent never touches storage.
3. **Outputs name the checkpoint they need**
   So the approval path is visible before you run anything.

PART 02 · UNDER THE HOOD

Academy — a worked example against seeded data

Given → it does → produces → decided by. Runnable in the demo as written.



Every agent ships with a worked example against the seeded fixtures, in four beats: given, it does, produces, decided by. These are runnable as written — the files named in "given" are in the artifact library when the demo boots. The lifecycle above it is the agent's declared `plan()`, with the tool and the rationale for each step.

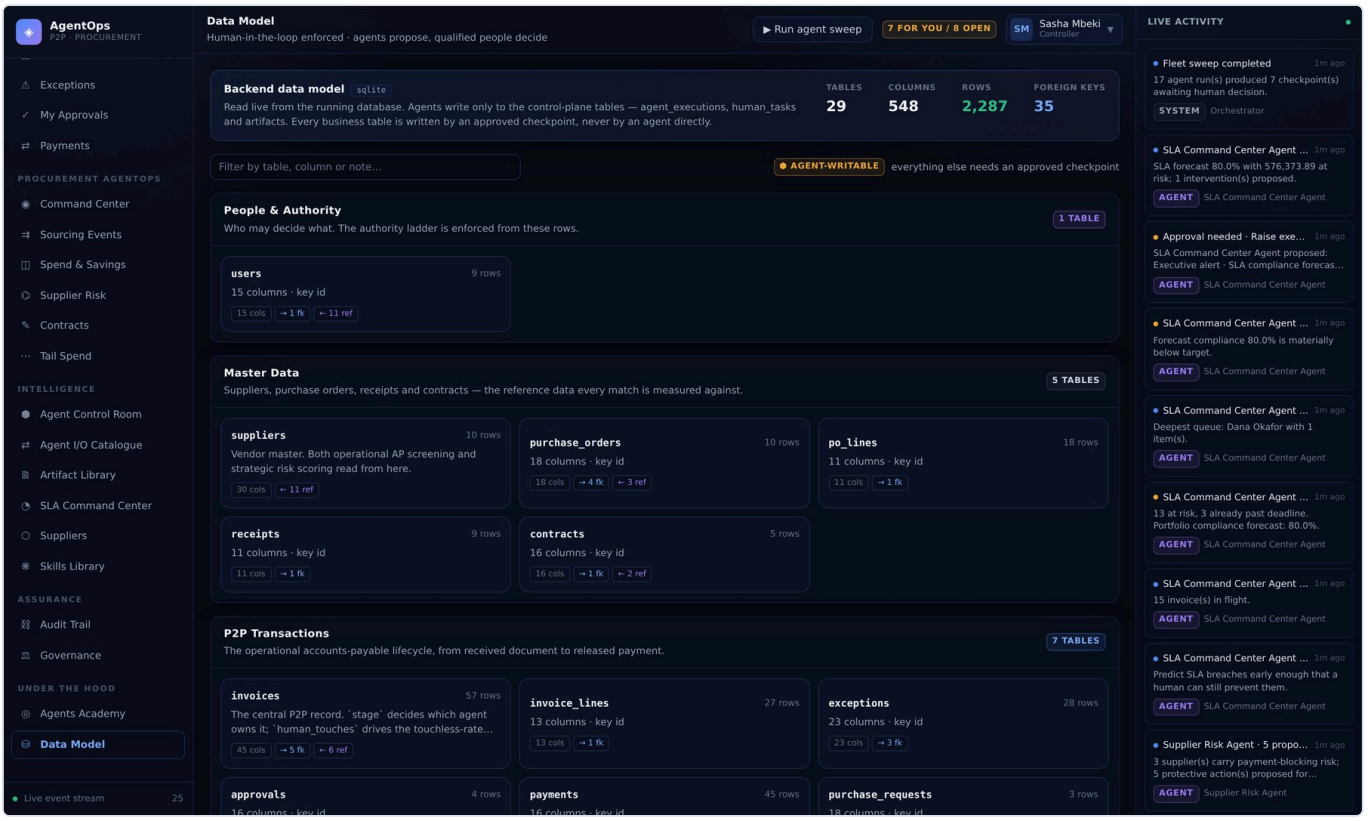
WHAT TO LOOK AT

1. **The lifecycle is the declared `plan()`**
   With the tool and the rationale for each step.
2. **The example uses shipped fixtures**
   You can reproduce it in the running demo in under a minute.
3. **"Decided by" names the authority**
   Including when two approvers are required.

PART 02 · UNDER THE HOOD

Data Model — the live schema, seven domains

29 tables introspected from SQLAlchemy metadata, with row counts read at request time.



The schema, live. Tables, columns, keys and foreign keys come from SQLAlchemy metadata; row counts are `COUNT(*)` executed when you open the page. Nothing is hand-maintained, so a new table appears the moment it exists — and a test fails if it was added without being placed in a domain. The three agent-writable tables are marked; every other table in the system is written only by an approved checkpoint.

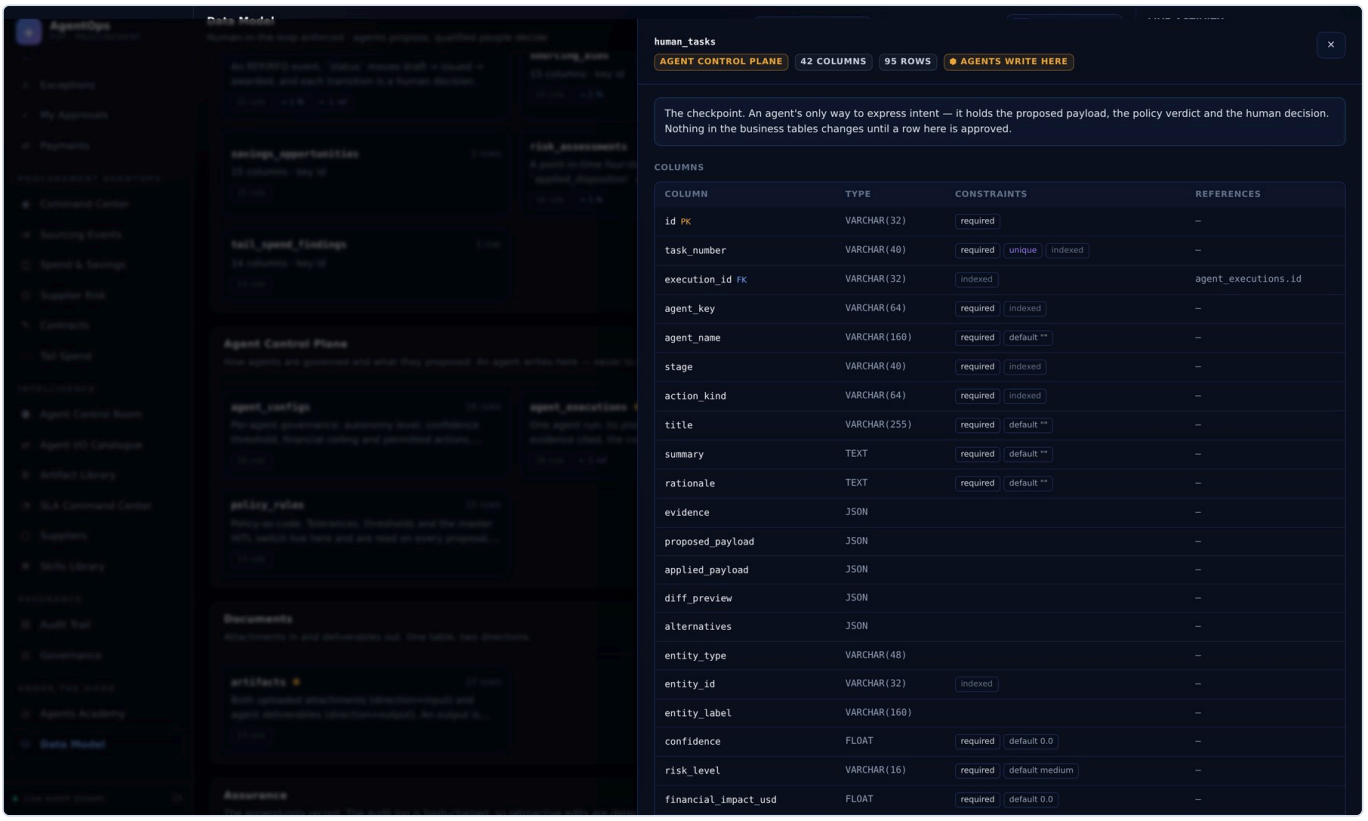
WHAT TO LOOK AT

1. **Agent-writable tables are marked**
   Only `agent_executions`, `human_tasks` and `artifacts`. Everything else needs an approved checkpoint.
2. **Nothing here is hand-maintained**
   A new table appears the moment it exists — and a test fails if it has no domain.
3. **Row counts are live**
   `COUNT(*)` per table per request. Cheap at demo scale; cache it at volume.

PART 02 · UNDER THE HOOD

Data Model — one table, both directions

Columns with types, keys, indexes and defaults, and foreign keys resolved both ways.



One table in full, with foreign keys resolved in both directions and clickable to jump. `human_tasks` is the one to read first — proposed payload, policy verdict, human decision and applied payload are all columns on the same row, which is the entire governance story in one place. Note the primary keys on `workflow_events` and `audit_logs`: they are integer PKs because SQLite only autoincrements a primary key, and both tables need a monotonic sequence.

WHAT TO LOOK AT

1. **`human_tasks` is the checkpoint**
   Proposed payload, policy verdict, human decision, applied payload — the whole story in one row.
2. **Both ends of every foreign key**
   Points-at and pointed-at-by, each clickable to jump.
3. **Watch the primary keys**
   `workflow_events` and `audit_logs` use an integer PK because SQLite only autoincrements a primary key.

PART 03

The code

The run loop, the anatomy of an agent, the policy engine, the executor registry — and how to add your own without breaking the guarantee.

PART 03 · THE CODE

The run loop

Identical for all sixteen agents. Everything except `plan()`, `gather()` and `decide()` is the base class — which is why the guarantees hold uniformly rather than per agent.

#	STEP	WHAT HAPPENS
1	**Resolve attachments**	`context["attachment_ids"]` → parsed records in `context["attachments"]`
2	**Check the config**	A disabled agent returns `blocked_by_policy` without running
3	**Plan**	`plan()` → `PlanStep`s, persisted to `agent_executions.plan`
4	**Execute / observe**	`gather()` → `Observation`s, each persisted and streamed live
5	**Reason**	`decide()` → `AgentDecision`; `llm.reason()` narrates what was already decided
6	**Escalate**	Records an event. It annotates — it does not change what a human decides
7	**Propose**	Each proposal → `policy.evaluate()` → a `HumanTask` carrying the verdict
8	**Report**	An audit entry and a completion event

Every step is persisted. Re-opening a run months later shows the plan it made, each tool and what came back, the evidence it cited, its confidence and the policy verdict on every proposal — which is what makes an agent decision auditable rather than merely logged.

Step 7 is where the guarantee lives. `policy.evaluate()` runs on every proposal, and the result is a `HumanTask` carrying the verdict — never a write. Auto-execution is reachable only when every gate opens, which the default configuration makes impossible.

Failure handling

An exception anywhere in the loop marks the execution `failed`, records the error on the row, emits a critical event and re-raises. Partial work is visible rather than discarded — the plan and any observations already made are persisted, so a failed run is still diagnosable.

PART 03 · THE CODE

Anatomy of an agent — `plan()`

Three-Way Match, the clearest of the sixteen. `plan()` declares the steps; it does not run them.

```
def plan(self, db: Session, context: dict) -> list[PlanStep]:
    return [
        PlanStep(1, "Fetch the purchase order and its lines", "po_fetch",
                 "The PO is the contractual basis for what may be billed."),
        PlanStep(2, "Fetch goods receipts posted against the PO", "gr_fetch",
                 "Receipt confirms the goods or services actually arrived."),
        PlanStep(3, "Run line-level variance analysis", "variance_analysis",
                 "Header totals hide offsetting line errors."),
        PlanStep(4, "Apply tolerance policy", "policy_compliance",
                 "Tolerance is a governance decision, not an agent preference."),
        PlanStep(5, "Propose clearance or an exception", "hitl_checkpoint",
                 "Either outcome is reviewed by AP before it takes effect."),
    ]
```

`plan()` **is called with an empty context** by the Agents Academy screen and by the build-specification generator. If it queried the database both would break — and it would no longer be a plan, it would be a trace.

Every step answers three things

FIELD	WHAT IT IS FOR
action	What the step does, written for a reviewer rather than as a log line.
tool	The name shown in the Academy, in the run timeline and in the live activity rail.
rationale	Why the step belongs. "Header totals hide offsetting line errors" is a reason a reader can judge; "fetch data" is not.

PART 03 · THE CODE

Anatomy of an agent — `gather()`

Collects evidence and returns one `Observation` per tool. Still writes nothing.

```
def gather(self, db: Session, context: dict) -> list[Observation]:
    store = PolicyStore(db)             # governance is data, read every run
    context["tolerances"] = {
        "amount_pct":   store.number("match.amount_variance_pct", 3.0),
        "quantity_pct": store.number("match.quantity_variance_pct", 2.0),
        "amount_abs":   store.number("match.amount_variance_abs", 50.0),
    }

    analysis = variance_analysis.analyze(
        invoice_lines=..., po_lines=..., received_qty_by_line=...,
        **context["tolerances"])
    context["analysis"] = analysis       # state on context, never on self

    return [Observation("variance_analysis",
                        f"Match '{analysis['match_result']}'",
                        analysis, ok=analysis["clean"])]
```

State on `context`, never `self`

Agents are singletons in the registry. Anything stored on the instance leaks into the next run — a bug that only appears under load or in a fleet sweep.

Policy is read per run

Tolerances come from `policy_rules` every time, so loosening one from the Governance screen takes effect on the next run with no deploy.

One Observation per tool

Each is persisted and streamed to the activity rail as it happens, so a reviewer sees what was asked and what came back — not just the conclusion.

Failure is an observation too

`ok=False` with a summary. An agent that cannot read its input reports that it could not, rather than raising and losing the run.

PART 03 · THE CODE

Anatomy of an agent — `decide()`

Returns an `AgentDecision`. Every field on it is something a reviewer will read.

```
def decide(self, db, context, observations) -> AgentDecision:
    analysis = context["analysis"]
    if not analysis["clean"]:
        ...                                  # exception path

    return AgentDecision(
        conclusion=f"{invoice.invoice_number} matches within tolerance.",
        confidence=0.97 if result == MatchResult.MATCHED else 0.94,
        decision_rules=[
            f"Amount variance {analysis['total_variance_pct']:+.2f}% "
            f"against a {tolerances['amount_pct']}% tolerance.",
        ],
        evidence=[evidence_item("Total variance", ..., "variance_analysis")],
        proposals=[ProposedAction(
            action_kind=ActionKind.ADVANCE_STAGE,
            payload={"invoice_id": invoice.id,
                     "stage": WorkflowStage.APPROVAL,
                     "match_result": result, "match_details": analysis},
            confidence=0.97, financial_impact_usd=amount)],
        handoff_to="approval_acceleration",
    )
```

Four rules for writing one

RULE	WHY
Confidence is a real number	One the agent stands behind. The policy engine gates on it, so a hard-coded 0.95 quietly defeats the control it is meant to feed.
Evidence cites its source	Every item names the observation it came from, so a reader can trace a claim back to the tool call that produced it.
Impact is what is actually at risk	It drives role escalation and dual approval. Inflated, it blocks routine work; deflated, it under-reviews real exposure.
Decision rules are quantitative	"+8.00% against a 3% tolerance" is a claim a supplier can dispute. "Looks wrong" is not.

Branch order matters. A blocking condition should return immediately rather than accumulating proposals — a suspected duplicate invoice is not worth correcting extraction fields on.

PART 03 · THE CODE

The policy engine — seven gates, default deny

`allow_auto_execute` requires *every* gate to open. Anything else becomes a checkpoint.

#	GATE	OPENS WHEN
1	**Global HITL enforcement**	`hitl.enforce_global` is off
2	**Agent autonomy**	The agent is at L3 or above
3	**Irreversibility**	The action is not in `IRREVERSIBLE_ACTIONS`
4	**Confidence**	Proposal confidence ≥ the agent threshold
5	**Financial envelope**	Impact ≤ the agent's `max_auto_amount_usd`
6	**Risk level**	Computed risk is not high or critical
7	**Action allow-list**	The action is in the agent's permitted set

```
# services/policy.py — every gate must open. Default deny.

if store.flag("hitl.enforce_global", settings.enforce_human_in_the_loop):
    blocked.append("Global human-in-the-loop enforcement is ON.")

if action_kind in IRREVERSIBLE_ACTIONS:
    blocked.append(f"'{action_kind}' is irreversible — always a human.")

if confidence < threshold:
    blocked.append(f"Confidence {confidence:.0%} below {threshold:.0%}.")

# Authority escalates with value, on top of the per-action minimum.
if financial_impact_usd >= 250_000:
    required_role, flag = Role.CFO, "cfo_threshold"
elif financial_impact_usd >= 50_000:
    required_role, flag = Role.CONTROLLER, "controller_threshold"

dual_approval = (financial_impact_usd >= dual_threshold
                 and action_kind in IRREVERSIBLE_ACTIONS)

return PolicyDecision(allow_auto_execute=not blocked, reasons=..., ...)
```

Turning global enforcement off opens gate 1 only. An L2 agent still stops, an irreversible action still stops, a low-confidence proposal still stops, and an over-ceiling amount still stops. The switch exists so the governance story is demonstrable end to end — not as a bypass.

Every gate that blocks records *why*, and that reason is stored on the task and rendered in the amber panel a reviewer reads. A checkpoint always explains itself, which is what makes the seventh gate — the action allow-list — safe to keep narrow.

PART 03 · THE CODE

The executor registry — the only writer

Thirty-three executors, one per action kind. If you are writing business data, you are writing one of these.

```
# services/hitl.py — the ONLY place business data is written.

_HANDLERS: dict[str, Handler] = {}

def action(kind: str):                        # registration decorator
    def wrapper(fn): _HANDLERS[str(kind)] = fn; return fn
    return wrapper

@action(ActionKind.ADVANCE_STAGE)
def _advance_stage(db, task, payload, actor) -> dict:
    invoice = _invoice(db, task, payload)
    before  = snapshot(invoice, INVOICE_AUDIT_FIELDS)
    _advance(db, invoice, payload["stage"], payload.get("status"), actor.full_name)
    write_audit(db, action="invoice.stage_advanced", actor=actor.full_name,
                actor_type="human", before_state=before,
                after_state=snapshot(invoice, INVOICE_AUDIT_FIELDS), ...)
    return {"stage": invoice.stage, "status": invoice.status}

# An action with no registered handler simply cannot happen.
```

Twelve actions are irreversible

```
post_to_erp · schedule_payment · release_payment · block_supplier
update_supplier_master · send_supplier_message · approve_purchase_request
issue_rfp · award_sourcing_event · issue_contract_for_signature
consolidate_suppliers · publish_executive_brief
```

The reasoning is uniform: each one reaches outside the system — a ledger, a bank, a supplier, a signature, a boardroom — and cannot be taken back by editing a row. None may execute without a person, at any autonomy level, with enforcement off.

An action with no registered handler simply cannot happen. A test asserts that every action an agent declares in `allowed_actions` has an executor, so an agent cannot advertise something the platform cannot apply.

Adding your own agent

Six steps. The test suite tells you what you forgot.

```
# 1. backend/app/agents/my_agent.py
class MyAgent(BaseAgent):
    key   = "my_agent"
    name  = "My Agent"
    suite = AgentSuite.P2P
    default_autonomy = AutonomyLevel.HUMAN_APPROVAL
    default_confidence_threshold = 0.90
    allowed_actions = [ActionKind.CREATE_EXCEPTION]
    inputs  = [IOSpec("invoice_id", "The invoice to inspect.", required=True)]
    outputs = [IOSpec("Checkpoint", "Open an exception — AP Clerk decides.",
                      kind="proposal")]

    def plan(self, db, context):    ...   # static declaration
    def gather(self, db, context):  ...   # one Observation per tool
    def decide(self, db, ctx, obs): ...   # returns proposals, never writes

# 2. registry.py     → add MyAgent to P2P_AGENT_CLASSES
# 3. explain.py      → add a WORKED_EXAMPLES["my_agent"] entry
# 4. agent_build_notes.py → add BUILD_NOTES["my_agent"]
# 5. python scripts/generate_agent_specs.py
# 6. pytest — three drift guards will tell you what you missed
```

The guards that catch you

IF YOU SKIP	WHAT FAILS
Registering an executor	An agent may not advertise an action nothing can apply.
The Academy lesson	A new agent must not ship undocumented.
The build notes	A specification without the reasoning is not one anyone can rebuild from.
Regenerating the spec	Specs are regenerated in memory and compared to what you committed.

Adding a new action kind

1. Add the member to `ActionKind` and a label to `ACTION_LABELS`.
2. Set its floor in `ACTION_MIN_ROLE`. If it reaches outside the system, add it to `IRREVERSIBLE_ACTIONS` — that is the decision that matters.
3. Write the executor in `services/hitl.py` under `@action(...)`. Snapshot before, mutate, `write_audit` with both states.
4. Add it to the proposing agent's `allowed_actions` and regenerate the specs.

PART 03 · THE CODE

Testing

79 tests. Three of them fail the build rather than the demo.

AgentOps — verification

```
$ cd backend && ../.venv/bin/python -m pytest -q
.....................................................................  [ 91%]
.......                                                                [100%]
79 passed in 9.65s

$ curl -H "X-User-Id: $PERSONA" localhost:8000/api/audit/verify
{
    "entries_checked": 368,
    "valid": true,
    "broken_at": null,
    "head_hash": "c4d9e0991bcf7f5d42ffdbd845bf63ddb7a9d3bf39282cb6a4a2ff208653d108"
}

$ curl -H "X-User-Id: $PERSONA" localhost:8000/api/data-model | jq .totals
{
  "tables": 29,
  "columns": 548,
  "rows": 2287,
  "relationships": 35
}
dialect: sqlite

$ python scripts/generate_agent_specs.py
Wrote 16 AGENT.md files + index to /home/user/AgentOps-EnterpriseAgentic-P2P-Platform/agents
$ python scripts/generate_skill_docs.py
Wrote 28 SKILL.md files + index to /home/user/AgentOps-EnterpriseAgentic-P2P-Platform/skills
```

Real output from the commands shown: the full suite, the audit chain replayed from genesis, the live schema totals, and both documentation generators.

```
def test_agent_cannot_mutate_state(seeded):
    before = db.execute(select(SavingsOpportunity)).all()
    get_agent("spend_analytics").run(db, {...})
    after = db.execute(select(SavingsOpportunity)).all()
    assert len(after) == len(before)


def test_spec_matches_the_code(agent_key):
    """Change an agent, forget to regenerate
       its spec, and this fails."""
    expected = gen.render(agent, BUILD_NOTES[key])
    committed = (ROOT / "agents" / key /
                 "AGENT.md").read_text()
    assert committed == expected
```

Three drift guards. A new table that lands outside a data-model domain; a new agent that ships without an Academy lesson or build notes; a committed `AGENT.md` or `SKILL.md` that no longer matches the code it describes. Each is a missed step rather than a bug — run the generators in `scripts/` and commit the result.

PART 04 · REFERENCE

Where to look when something is wrong

SYMPTOM	WHERE TO LOOK
An agent proposes nothing	Its `decide()` returned no proposals, or the agent is disabled in `agent_configs`. Check the run in the Agent Control Room — a blocked run says so explicitly.
A proposal will not execute	The checkpoint's **Policy & audit** tab names the gate that held it and the role required. That is the verdict itself, not a reconstruction.
A decision is refused	Authority. `ACTION_MIN_ROLE`, plus value-based escalation, plus dual approval above the threshold on irreversible actions.
The audit chain is broken	`GET /api/audit/verify` names the first divergent row. Something wrote history out of band.
A deliverable stays a draft	Its checkpoint was rejected, or the proposal never carried `artifact_ids` — only a linked artifact is released on approval.
A documentation test fails	Not a bug, a missed step. Run `scripts/generate_agent_specs.py` and `scripts/generate_skill_docs.py`, then commit.
Port 8000 is busy	`lsof -ti :8000 | xargs kill`, or edit the `uvicorn.run(...)` call in `backend/app/main.py`.
The UI looks stale in --dev	You are on port 8000, which serves the last built SPA. Use port 5173.

Read next, in this order

1. `docs/SPECIFICATION.md` — the platform: guarantees, enumerations, data model, governance, API, and a rebuild order.
2. `agents/<key>/AGENT.md` — one complete build specification per agent, enough to rebuild it without the source.
3. `skills/<name>/SKILL.md` — the 28 shared capabilities and their contracts.
4. The **Agents Academy** screen — the same material, live, against seeded data you can actually run.

> **Check any rebuild against the invariant, not the feature list.** An agent that can write to a business table gives you the same screens and none of the guarantees.